
DISTRIBUTION-INDEPENDENT ESTIMATOR FOR THE TRAVELING SALESMAN PROBLEM ON ARBITRARY DIMENSIONS

Stephen Feldman, James P. Bailey, Bahar Cavdar
Rensselaer Polytechnic Institute, Troy, NY
{feldms5, bailej6, cavdab2}@rpi.edu

ABSTRACT

We present Geometrically Assisted Regression Tree (GART) 2.0, a distribution-independent estimator of optimal Traveling Salesman Problem (TSP) tour length whose feature set scales to high dimensions. Existing estimators either require knowledge of the underlying distribution or are not scalable to high dimensions. GART 2.0 is a LightGBM regressor that predicts a scaling ratio α , the relationship between the length of the minimum spanning tree and the optimal tour length. We benchmark GART 2.0 against its predecessor and other tour-length estimators from the literature across three datasets. Benchmarking demonstrates that GART 2.0 outperforms the surveyed models while remaining computationally tractable. Because its features scale well with dimension, GART 2.0 can also predict tour lengths for non-Euclidean data. Using multidimensional scaling, the model processes non-Euclidean TSPLIB95 instances translated to high-dimensional Euclidean spaces and makes high-quality estimates. Our results demonstrate that the GART framework can be applied to real-world problems.

Keywords Traveling Salesman Problem · Tour length estimation · Distribution-independent · Multidimensional network

1 Introduction

The Traveling Salesman Problem (TSP) is a fundamental problem in combinatorial optimization, widely used for logistics operations planning, network design, and resource allocation [Chien, 1992, Applegate et al., 2006]. Because determining exact solutions for large-scale instances is NP-hard [Karp, 1972, Papadimitriou, 1977], the ability to compute rapid tour-cost estimates is essential for real-time decision-making in applications where a full solve is impractical. Tour-cost estimates also serve as inputs to other optimization problems where the TSP appears as a subproblem. For example, in the mobile facility location problem [Halper and Raghavan, 2011], the objective function includes the cost of a TSP route through customer locations.

1.1 Background and Motivation

Tour-length estimation is a key input to strategic logistics decisions where solving the TSP exactly is unnecessary or infeasible. Last-mile delivery accounts for approximately 53% of total shipping costs [Finmile, 2025], and the UPS ORION system is reported to save roughly 100 million miles and \$300–400 million annually through route optimization [Holland et al., 2017]. As an extension of the Vehicle Routing Problem (VRP), last-mile delivery solvers use complex heuristics to construct efficient routes. In such systems, recalculation occurs frequently as new orders are added to the system, and the number of customers served by a route is high. Distributors are looking for operational solutions that offer efficient routes without the computational expense of traditional solves. Tour length estimators enable significant computational improvements without sacrificing accuracy.

Beyond traditional logistics, fast TSP length approximation is a critical component across diverse applications in which TSP-like structures arise as subproblems. In bioinformatics, the ordering of markers in radiation hybrid mapping is modeled as a TSP, where rapid cost estimation enables evaluation of assembly quality without full reconstruction [Agarwala et al., 2000]. In data compression, vector quantization partitions a codebook into clusters and must order codewords to minimize transition costs, a problem reducible to TSP on a high-dimensional distance matrix. Additionally, in sensor network design, estimating the minimum data collection path length is crucial for optimizing the lifespan of battery-constrained mobile sinks in wireless sensor networks [Wang et al., 2008].

In these domains, the problem is rarely confined to 2D Euclidean space. Genome assembly operates in abstract metric spaces defined by overlap similarity [Agarwala et al., 2000], while sensor networks involve routing through high-dimensional feature spaces [Wang et al., 2008]. While a full evaluation on these application-specific metrics is beyond the scope of this study, they motivate the need for an estimator that generalizes beyond 2D Euclidean geometry. Furthermore, metaheuristics for large-scale routing frequently partition problems into smaller sub-TSPs, each requiring a rapid cost estimate to guide the search [Mariescu-Istodor and Fränti, 2021, Pan et al., 2023]. For example, Pan et al. [2023] decompose large TSP instances hierarchically, solving each cluster as an independent sub-problem. A fast, accurate tour-length estimator enables such algorithms to evaluate candidate partitions without solving each sub-TSP exactly. In these selection tasks the value of an estimator lies less in any single tour-cost number than in its ability to order candidates the way the true optima would: a planner choosing among alternative node sets, route splits, or cluster assignments needs the cheaper option to be predicted as cheaper. We therefore ask GART 2.0 to be not only accurate but *rank-preserving*, and we test this property directly in Section 4.

Despite this breadth of applications, most classical tour-length estimators are built for low-dimensional Euclidean inputs and depend on geometric quantities—most often convex-hull area—that become unstable or intractable as dimension grows; convex-hull computation alone scales exponentially with dimension [Chazelle, 1993]. An estimator useful across these settings must therefore work well beyond 2D Euclidean TSP. This motivates a dimension-scalable feature set: because many real, non-Euclidean instances can be embedded into Euclidean space with acceptable distortion once enough dimensions are allowed, an estimator that stays accurate at those dimensions can be applied to them as well. Our approach replaces the convex-hull enclosure with a combination of MST topology and lightweight statistics, and we demonstrate that it yields lower error than the surveyed estimators on both synthetic and real-world TSPLIB95 benchmarks.

1.2 Limitations of the Current Models

While the need for a TSP estimation model for general cases is clear, a persistent gap remains in the literature regarding high-dimensional or non-Euclidean instances. Traditional estimators such as the BHH theorem [Beardwood et al., 1959] and the probabilistic model of Chien [1992] are formulated for arbitrary dimensions, but in practice their calibrating constants and the convex-hull volume on which they depend are reliable only in low dimensions. Embedding a non-Euclidean instance into Euclidean coordinates is possible but requires a high-dimensional target space to avoid distortion [Bourgain, 1985], which puts those estimators back into the regime where their geometric inputs are unstable. A parallel line of work develops regression-style estimators tuned to 2D Euclidean instances: Daganzo [1984] relates tour length to the number of points and the enclosing area, Kwon et al. [1995] fit regression and neural models on area- and dispersion-based predictors, and Çavdar and Sokol [2015] propose a distribution-free estimator built from per-axis spread statistics. Space-filling heuristics such as the Hilbert-sort construction of Bartholdi and Platzman [1982] instead estimate tour length in $O(n \log n)$ with no solve at all. Each of these depends on convex-hull area or an axis-aligned bounding box, both of which degrade in higher dimensions; we benchmark GART 2.0 against all of them in Section 4 (Table 1).

Recent work has attempted to escape these constraints with neural networks. Varol et al. [2024] reports sub-1% deviation on standard distributions by combining an MLP with domain-knowledge features, but their pipeline requires

solution-level features, which are computationally expensive and challenging to generalize across all instance sizes. Kou et al. [2022] use the standard deviation of random feasible tour lengths as a single-feature predictor, and evaluate it on both Euclidean and non-Euclidean instances. The predictor is distribution-agnostic but requires repeated tour sampling, which raises its per-instance cost. Kou et al. [2024] extend this randomization principle beyond the TSP, predicting optimal values for the VRP, MST, and matching problems from the standard deviation of random feasible solutions; like the original predictor it relies on repeated per-instance sampling, whose cost we exclude on the same grounds (Section 4.1). The most direct methodological neighbor is Kou et al. [2023], which pairs a Sammon-map embedding with a distribution-independent estimator on non-Euclidean TSPLIB95 instances; our pipeline differs by using classical metric MDS for the embedding while computing MST-topology features on the original distance matrix rather than the embedded coordinates.

1.3 Contribution: The GART 2.0 Model

We present the Geometrically Assisted Regression Tree (GART 2.0) model, a LightGBM-based regressor [Ke et al., 2017] that learns to predict a per-instance scaling factor $\hat{\alpha}$ rather than using a fixed constant. It is distribution-independent: it requires no knowledge of the underlying distribution as input, although performance can vary across distribution type. Distribution-independence is not by itself new—prior estimators such as Çavdar and Sokol [2015] and Kou et al. [2022, 2023] are also distribution-independent—but those models remain tied to 2D coordinate geometry or to per-instance embeddings. The contribution of GART 2.0 is to be distribution-independent *and* dimension-scalable at the same time: every feature is defined for arbitrary dimension d through MST topology, and the same feature set applies unchanged as both the dimension and the instance size grow, so no geometric input degrades as the problem leaves the 2D regime. To achieve this we replace features that scale poorly with dimensionality, such as convex hulls and principal component analysis, with MST-derived topological statistics and lightweight coordinate-range descriptors. We benchmark our estimator against surveyed models in the literature on three datasets including TSPLIB95. We compare favorably against the surveyed models, and demonstrably improve on the fixed MST-ratio baseline. Furthermore, we show GART 2.0 performs well on non-Euclidean data from TSPLIB95, using multidimensional scaling to convert the data to high-dimensional Euclidean space. GART 2.0’s features are either computed from the MST or bounded by the MST’s complexity; combined with its small inference time, this lets GART 2.0 outperform exact solves on medium and large instances.

1.4 Paper Outline

The remainder of this paper is structured as follows. Section 2 introduces the theoretical foundations of the multiplicative MST–TSP relationship and the Christofides–Serdyukov bound [Christofides, 1976, Serdyukov, 1978]. Section 3 describes GART 2.0’s feature set, dataset generation, and implementation. Section 3 also covers model training with hyperparameter tuning via Optuna [Akiba et al., 2019] and complexity analysis. Section 4 defines the baseline estimators, introduces the three benchmark datasets, reports per-dataset results, and discusses the patterns they reveal. Section 5 extends GART 2.0 to non-Euclidean TSPLIB95 instances via MDS embedding. Section 6 summarizes contributions, limitations, and future directions.

2 Theoretical Foundations

To formally situate the estimation problem, we first define the Traveling Salesman Problem (TSP) using its classic integer linear programming (ILP) formulation. Let $G = (V, E)$ be a complete undirected graph where $V = \{1, \dots, n\}$ is the set of vertices and $E = \{(i, j) : i, j \in V, i < j\}$ is the set of edges. Associated with each edge (i, j) is a non-negative cost c_{ij} , typically representing the Euclidean distance between points in $\mathbb{R}^d$. The objective is to find a tour of minimum total cost that visits every vertex exactly once. Following the standard Dantzig-Fulkerson-Johnson formulation of Applegate et al. [2006], we use binary decision variables x_{ij} , where $x_{ij} = 1$ if edge (i, j) is included in

the tour, and 0 otherwise. The problem formulation is as follows:

$$\min \sum_{i=1}^{n-1} \sum_{j=i+1}^n c_{ij} x_{ij} \quad (1)$$

Subject to the degree constraints:

$$\sum_{j < i} x_{ji} + \sum_{j > i} x_{ij} = 2, \quad \forall i \in V \quad (2)$$

And the sub-tour elimination constraints:

$$\sum_{i \in S} \sum_{j \in S, j > i} x_{ij} \leq |S| - 1, \quad \forall S \subset V, 2 \leq |S| \leq n - 1 \quad (3)$$

The TSP is NP-hard [Karp, 1972]. This formulation establishes the combinatorial structure we exploit: any feasible tour is a 2-regular spanning subgraph, of which the MST is the cheapest connected relaxation.

In contrast to the intractable TSP, the Minimum Spanning Tree (MST) problem, which seeks a connected, acyclic subgraph minimizing the total edge weight, can be solved optimally in polynomial time. Classical algorithms such as Prim’s [Prim, 1957] and Kruskal’s construct the MST in $O(n^2)$ time for complete graphs. When nodes are situated in $\mathbb{R}^d$, the pairwise distance computation scales as $O(n^2 \cdot d)$, making the total cost linear in d and thus a computationally stable baseline for high-dimensional inference.

The relationship between these two problems bounds the optimal TSP tour cost, denoted L_{TSP} , between one and two times the MST weight. The lower bound is immediate: removing the heaviest edge from any tour leaves a spanning tree, so $L_{\text{MST}} \leq L_{\text{TSP}}$. The upper bound, $L_{\text{TSP}} \leq 2 L_{\text{MST}}$, holds for any instance obeying the triangle inequality; it is the standard corollary of the double-tree heuristic [Rosenkrantz et al., 1977], which doubles the MST edges into an Eulerian multigraph and shortcuts repeated vertices to yield a feasible tour of length at most $2 L_{\text{MST}}$. Together these give

$$L_{\text{MST}} \leq L_{\text{TSP}} \leq 2 \cdot L_{\text{MST}}. \quad (4)$$

The upper constant of 2 cannot be reduced in general: for collinear point sets the optimal tour must traverse the line to its far end and return, so $L_{\text{TSP}} = 2 L_{\text{MST}}$ exactly.

A separate and frequently conflated result concerns *algorithmic* approximation rather than this intrinsic ratio. The Christofides–Serdyukov algorithm [Christofides, 1976, Serdyukov, 1978] augments the MST with a Minimum Weight Perfect Matching (MWPM) on the odd-degree vertices to produce a tour guaranteed to be within a factor $\frac{3}{2}$ of the *optimum*, i.e. $L_{\text{alg}} \leq \frac{3}{2} L_{\text{TSP}}$ (recently sharpened to $\frac{3}{2} - \epsilon$ by Karlin et al. [2021]). This is a guarantee on an algorithm’s output relative to the optimal tour; it does not tighten the intrinsic ratio $L_{\text{TSP}}/L_{\text{MST}}$, which can still equal 2. Computing the MWPM moreover requires $O(n^3)$ time, far above the $O(n^2)$ MST construction, so the Christofides refinement is impractical as a fast estimation subroutine.

We therefore treat the multiplier $\alpha = L_{\text{TSP}}/L_{\text{MST}} \in [1, 2]$ as an intrinsic, instance-dependent quantity to be predicted, rather than something to bound with a fixed constant or recover with an expensive algorithm. In practice α is well below its worst-case value of 2, and its expected value depends on both the dimensionality d and the instance size n . Percus and Martin [1996] showed that for uniformly distributed points, α decreases as d increases: the MST and TSP lengths converge because higher-dimensional spaces offer more shortcuts for the tour to exploit relative to the tree. This implies that the computational gap between the MST and the optimal TSP tour diminishes as dimensionality increases. Consequently, a static estimator calibrated for 2D (where $\alpha \approx 1.075$ as the ratio $\beta_{\text{TSP}}(2)/\beta_{\text{MST}}(2) \approx 0.7124/0.663$ of the classical BHH constant [Johnson et al., 1996] and the empirical 2D MST constant [Cortina-Borja and Robinson, 2000]) will systematically overestimate tour costs in higher dimensions. Our proposed model addresses this by learning to predict α dynamically based on the topological characteristics of the vertex set.

3 Methodology: The Geometrically Assisted Regression Tree

In one sentence, this section builds a model that estimates the ratio between the minimum-spanning-tree length and the optimal tour length. It then recovers the tour length by multiplying that ratio by the (cheaply computed) MST length. The model is a *regression tree*: a decision tree used to estimate a numeric value from a set of predictors, or *features*. Starting at the root, the tree asks whether a feature meets some threshold (for example, “is the largest MST edge more than twice the average?”) and follows the matching branch; this repeats until a leaf node is reached, where the tree outputs its estimate of the ratio. In practice we do not use a single tree but an *ensemble* of many trees whose estimates are summed, which is more accurate than any one tree; the details of that ensemble appear in Section 3.3.

The remainder of this section can be skipped by a reader willing to take the estimator as a black box. For everyone else, it proceeds in four parts: (i) the features computed from each instance and given to the trees (Section 3.1); (ii) the data used to train the model and grow the trees (Section 3.2); (iii) the modeling choices and complexity analysis that lead to the final ensemble (Section 3.3); and (iv) the resulting feature importance and model validation (Section 3.4).

We enhance the proposed Geometrically Assisted Regression Tree from [Feldman, 2025] to create GART 2.0, a framework that estimates the TSP tour cost using the relationship between the MST and the optimal tour. GART 2.0 integrates gradient-boosted regression trees with a dimension-agnostic feature set derived from the MST topology to overcome the dimensional limitations of classical geometric estimators. Unlike traditional approaches that map coordinate-dependent metrics directly to tour cost, GART 2.0 decouples the estimation into a deterministic MST computation followed by a statistical prediction of the scaling coefficient $\alpha = L_{\text{TSP}}/L_{\text{MST}}$. The final estimator is $\hat{L}_{\text{TSP}} = \hat{\alpha} \cdot L_{\text{MST}}$, where $\hat{\alpha}$ is predicted by a LightGBM regressor based on 30 topological features of the instance.

We constrain the regression target to $\alpha \in [1.0, 2.0]$. The lower bound is feasibility-exact: every TSP tour is a spanning structure, so $L_{\text{TSP}} \geq L_{\text{MST}}$. The upper bound is the double-MST worst-case ratio, and in all training data we observe α well inside this interval. Predictions are therefore clipped to $[1.0, 2.0]$ at inference. The following section introduces the feature set.

3.1 Feature Selection

To predict the scaling factor α with high precision across varying dimensions, we extract 30 features from each TSP instance. The feature count reflects the need to characterize three distinct aspects of instance structure—spatial spread, edge-weight distribution, and branching topology—each requiring multiple statistics to capture the range of behaviors observed across distributions. Our features combine MST edge weights and degree sequence, which are invariant to rotation, translation, and coordinate-system choice, with lightweight geometric descriptors such as bounding hyper-volume and centroid distances that generalize naturally to arbitrary dimensions. The dimensionality d enters the feature set as an explicit input rather than implicitly through coordinate geometry.

Before any of the geometric descriptors are evaluated, we rotate the input coordinates into their principal-axis frame [Jolliffe, 2002]: the covariance matrix of the centered coordinates is diagonalized by a symmetric eigen-decomposition, the eigenvectors are ordered by descending variance, and the sign of each axis is fixed by the coordinate of largest absolute magnitude so the frame is deterministic under reflections. This is the d -dimensional analogue of aligning a 2D point set with the minimum-area bounding rectangle, and it costs $O(nd^2 + d^3)$ which is dominated by the MST construction it precedes. The rotation leaves the MST itself invariant, because every edge weight is a Euclidean distance and Euclidean distances are preserved under rotation; it also leaves the centroid dispersion statistics invariant, because those are functions of distances to the mean. What it does change is the axis-aligned bounding box, thereby removing orientation noise from whatever frame the instance happens to arrive in.

The features fall into three groups. Geometric and centroid descriptors quantify spatial spread and density. MST edge distribution statistics capture local uniformity through edge weight moments. Topological structure and clustering proxies detect higher-order patterns such as clusters and filaments.

3.1.1 Geometric and Centroid Descriptors

The first category quantifies the macroscopic spatial properties of the node set in $\mathbb{R}^d$. These features provide the model with scale information analogous to the volume term V in the BHH formula $\hat{L} = \beta_d \cdot n^{(d-1)/d} \cdot V^{1/d}$ [Beardwood et al., 1959], but computed from coordinate ranges rather than convex hull volume, thereby bypassing the computational complexity of computing volume in higher dimensions. For example, centroid-distance statistics, adapted from Çavdar and Sokol [2015], measure the dispersion of nodes relative to the geometric center, enabling the model to distinguish uniform distributions from centralized Gaussian clusters. Table 13 in Appendix J lists these features.

3.1.2 MST Edge and Topological Features

The remaining 22 features summarize the MST itself. Eleven describe the edge-weight distribution: mean, standard deviation, skewness, kurtosis, max, total length, and the 10/25/50/75/90 quantiles. These are invariant to rotation and translation. The higher moments act as scale-mixture detectors that move α sharply when the point set contains dense clusters separated by voids. The other eleven describe topology: dominance, gap, and leaf ratios; degree mean, standard deviation, and maximum; large-edge count; and an $O(n)$ MST diameter (with its normalized form) computed via double-BFS. The topology block proxies cluster, filament, and hub structure without an explicit clustering pass. Per-feature definitions, complexities, and citations are tabulated in Appendix C (Tables 6, 7).

3.2 Dataset Generation

To train a general model, we constructed a synthetic dataset with topological diversity across four distribution classes: Uniform, Normal (Gaussian), Clustered (k -means centers), and Correlated (autoregressive across dimensions). For each instance, dimensions are independently assigned a distribution class, producing anisotropic configurations where the topology varies by axis.

The full feature corpus comprises 106,272 solved instances with node counts $n \in [5, 1000]$ and dimensions $d \in \{2, 3, 4, 5, 6, 7, 8, 9, 10, 15, 20, 25, 30, 35, 40, 45, 50\}$ (17 training-visible dimension values), plus a held-out extrapolation bucket at $d = 100$. Instances are partitioned into training (69,768), validation (19,584), and test (16,920) via a stratified split on $(d, n, \text{grid size})$; all $d = 100$ instances are locked into the test partition so the model never sees that dimension during training or validation. The full training grid is given in Appendix B (Table 5); extrapolation to $d = 100$ is evaluated in Section 4.3.1.

For each instance we computed the ground-truth tour by running both Concorde [Applegate et al., 2006] and LKH-3 [Helsgaun, 2017] and retaining the shorter of the two tours. Concorde is an exact solver: it returns a provably optimal tour together with a matching lower bound that certifies optimality. LKH-3 is a state-of-the-art heuristic that returns a high-quality tour but no optimality certificate. On every training instance ($n \leq 1000$) Concorde certified optimality within its computational budget, and LKH-3 independently returned a tour of the same length; the two solvers therefore agreed, and we take that verified optimum as ground truth. (Running both is a safeguard: the shorter-tour rule lets a strong heuristic solution stand in on any instance Concorde cannot certify, although no such instance arose in the training set.) Coordinate scaling was applied to preserve floating-point precision in the integer-valued TSPLIB format required by these solvers.

3.3 Model Architecture and Training

The estimator for α is a *gradient-boosted ensemble* of the regression trees introduced at the start of this section. Two ideas motivate this choice. First, a single regression tree is easy to interpret but high-variance, so combining many trees yields a far more stable estimate. Random forests [Breiman, 2001] do this by averaging trees grown independently on bootstrap samples of the data. Gradient-boosted decision trees (GBDT) [Friedman, 2001] instead grow the trees *sequentially*: each new tree is fit to the error left over by the trees before it, so the ensemble keeps improving where it is

currently weakest. We use LightGBM [Ke et al., 2017], a fast and memory-efficient GBDT implementation, as the regressor; its final prediction $\hat{\alpha}$ is the sum of all tree outputs.

We selected LightGBM over standard random forests or other GBDT implementations because of its leaf-wise tree-growth strategy. Unlike level-wise algorithms that grow trees depth-first to keep them balanced, LightGBM splits whichever leaf promises the largest reduction in loss, regardless of depth. This is well suited to approximating α because the relationship between graph topology and tour cost often exhibits sharp, non-smooth transitions—for example, α can fall steeply once a point set coalesces into distinct clusters. Leaf-wise growth lets the model spend its capacity on these critical transition regions rather than on splits that merely re-describe the bulk of the training distribution.

We train the model to minimize the Root Mean Squared Error (RMSE) between the predicted ratio $\hat{\alpha}$ and the ground-truth α , and clip $\hat{\alpha}$ to the theoretical interval $[1.0, 2.0]$ from Equation 4 at inference. In practice this clip never activates: every one of the 16,920 predictions on the multi-dimensional test set already falls inside $[1.031, 1.948]$, comfortably within the admissible interval. To optimize the model’s generalization capability, we employed the Optuna framework [Akiba et al., 2019] to perform a Bayesian search of the hyperparameter space. The optimization utilized the Tree-structured Parzen Estimator (TPE) algorithm over 100 trials to tune the learning rate, tree structure, and regularization terms. The final model configuration, detailed in Table 14 of Appendix K, resulted in an ensemble of 2,031 trees. While the hyperparameter search allowed for up to 300 leaves per tree to capture complex interactions, the final trained model exhibited a maximum leaf count of 148 and an average of 148.0 leaves per tree, indicating that the constraint was tight enough to prevent overfitting.

Note that inference cost is dominated by MST construction, not tree traversal. The trained model performs $K = 2,031$ tree traversals at inference, with an average root-to-leaf depth of $\bar{D} \approx 18.9$ (empirical maximum 33). Since LightGBM grows leaf-wise, depth is not $\lceil \log_2 L \rceil$ but $K \cdot \bar{D} \approx 3.8 \times 10^4$ comparisons, independent of n . Feature extraction is dominated by the MST: $O(n^2 \cdot d)$ in general (dense distance matrix), reduced to $O(n \log n)$ in 2D by constructing the MST on a Delaunay-sparse edge set [Delaunay, 1934], since the Euclidean MST is a subgraph of the Delaunay triangulation. Sorting the $n - 1$ MST edges for quantile features adds $O(n \log n)$, and the remaining topological statistics (degrees, diameter via two BFS passes, leaf counts) are $O(n)$. The total asymptotic complexity is therefore $O(n^2 \cdot d)$ in general, and $O(n \log n)$ for 2D inputs. For comparison, the Christofides–Serdyukov $\frac{3}{2}$ -approximation requires a Minimum Weight Perfect Matching (MWPM) on the odd-degree MST vertices, with exact algorithms in $O(n^3)$ [Edmonds, 1965, Gabow, 1990, Cook and Rohe, 1999]; approximate MWPM variants reduce this but sacrifice the 1.5 guarantee [Duan and Pettie, 2014, Drake and Hougardy, 2003], and Christofides returns a worst-case upper bound rather than a point estimate.

3.4 Feature importance and design validation

We validate the 30-feature set with SHAP values [Lundberg and Lee, 2017] on the held-out test split. The MST dominance ratio alone carries 26.4% of the model’s decision mass, and six of the top ten features are MST-derived (dominance, degree mean, diameter, normalized diameter, gap, large-edge count), consistent with the BHH intuition that tour length scales with tree-like spanning structure. Dimension and node count contribute 22.2% of the decision mass jointly—the channel through which the model adapts to varying problem scale without a distribution label. Centroid-distance dispersion (9.9% summed across the four descriptors) and bounding hypervolume ($\sim 4.6\%$ summed across log and linear forms) contribute meaningfully once evaluated in the canonical principal-axis frame (§3.1). Adding further hand-engineered features (e.g. a greedy nearest-neighbor tour length) caused the model to overfit to the training distributions rather than generalize. The full 30-feature ranking is reported in Appendix D (Table 8).

4 Benchmarking

We evaluate GART 2.0 against established estimators on three benchmarks: a synthetic 2D dataset spanning five distribution classes, a multi-dimensional (ND) synthetic dataset covering $d \in \{2, 3, \dots, 50, 100\}$ (with $d = 100$ reserved as an extrapolation bucket outside the training range), and the 78 EUC_2D instances of TSPLIB95 [Reinelt, 1991]. We use “ND” as shorthand for this multi-dimensional benchmark throughout. The 2D suite mixes uniform and non-uniform generators: anisotropic clusters, geometric lattices, and near-linear cases. The near-linear generator was not part of the training distribution; we include it explicitly to probe how MST-based estimation handles geometries outside what the model was taught.

Throughout this evaluation, we report the Standard Deviation of Percent Error (SDPE) as the primary metric (first column in every results table), with MAPE and median absolute percentage error reported alongside for completeness to provide a measure of accuracy. The SDPE measures precision and is defined as:

$$\text{SDPE} = 100 \cdot \sqrt{\frac{1}{N-1} \sum_{i=1}^N (e_i - \bar{e})^2}, \quad \bar{e} = \frac{1}{N} \sum_{i=1}^N e_i.$$

We accompany each SDPE with a 95% bootstrap confidence interval (1000 resamples). MAPE measures average error; median absolute percentage error is reported alongside as an outlier-robust companion.

4.1 Benchmark Models

We benchmark GART 2.0 against the estimators listed in Table 1, which gives each baseline together with its formula, asymptotic cost, and source reference: seven classical estimators, plus our own predecessor GART 1.0 [Feldman, 2025]. GART 1.0 serves as a controlled comparison—it shares GART 2.0’s learning target but uses the older 2D-specific feature set—so the gap between the two isolates the contribution of the new MST-derived features rather than of the learning approach itself. We exclude estimators whose per-instance cost equals or exceeds the optimal solver’s at the benchmarked size: this rules out Christofides–Serdyukov [Christofides, 1976, Serdyukov, 1978] at $O(n^3)$ MWPM cost, the random-tour-sampling family [Basel and Willemain, 2001, Kou et al., 2022, 2024] whose sampling cost approaches solver time at $n \geq 100$, and Varol et al. [2024] (whose pipeline requires solution-level features). Table 1 groups the included baselines by applicability: only the Hilbert curve and the MST Ratio are dimension-agnostic and appear in the multi-dimensional results; BHH, Çavdar–Sokol, Kwon, Daganzo, Chien, and GART 1.0 all rely on 2D-specific geometry (convex hull or axis-aligned bounding rectangle) and are evaluated only on the 2D synthetic and TSPLIB EUC_2D benchmarks. Per-baseline implementation notes (e.g. Kwon’s coordinate-rescaling fix, the Çavdar–Sokol small- n correction, the Hilbert-versus-Sierpinski lineage) are recorded in Appendix E.

GART 1.0 is the direct predecessor of the present model and shares its goal of learning α ; as noted above, it is included to isolate the effect of GART 2.0’s new feature set, not as an independent peer-reviewed benchmark. Because GART 1.0’s feature extractor relies on 2D-only primitives (Graham-scan hull, 2D PCA), it appears only on the 2D synthetic and TSPLIB EUC_2D benchmarks (Tables 11, 12). The MST Ratio baseline estimates the tour as $\rho(d) L_{\text{MST}}$, where $\rho(d)$ is the expected $L_{\text{TSP}}/L_{\text{MST}}$ ratio at dimension d . Percus and Martin [1996] established only its limiting value, $\rho(d) \rightarrow 1$ as $d \rightarrow \infty$; the values at finite d are not given in closed form, so we supply our own monotone schedule that interpolates between a calibrated 2D anchor and that limit. Concretely, for BHH and MST Ratio we use $\beta_2 = 0.7124$ [Johnson et al., 1996] and $\rho(2) = 1.075$ computed as $\beta_{\text{TSP}}(2)/\beta_{\text{MST}}(2)$ with $\beta_{\text{MST}}(2) \approx 0.663$ [Cortina-Borja and Robinson, 2000]; the full $\rho(d)$ schedule for $d \geq 3$ is reported in Appendix E. BHH is reported only for $d = 2$ because the hull-to-bounding-box volume ratio decays exponentially with dimension, making V unstable as an estimator at $d \geq 3$ [Carlsson and Yu, 2024].

Table 1: Benchmark estimators compared against GART 2.0. A is convex-hull area, V bounding hypervolume, R mean centroid distance, n instance size, d dimension. Formula columns omit dimension-correction constants given in Appendix E.

Estimator	Formula	Cost	Source
<i>Dimension-agnostic baselines (evaluated on 2D, ND, and TSPLIB EUC_2D)</i>			
Hilbert curve	$\sum \ x_{(i)} - x_{(i+1)}\ $ on Hilbert-sorted points	$O(n \log n)$	Bartholdi and Platzman [1982]
MST Ratio	$\rho(d) L_{\text{MST}}$	$O(n^2 d)$	$d \rightarrow \infty$ limit [Percus and Martin, 1996]; finite- d schedule ours
<i>2D-only baselines (evaluated on 2D and TSPLIB EUC_2D; omitted from ND)</i>			
GART 1.0	2D-only feature set, GBDT regression on α	$O(n \log n)$	Feldman [2025]
BHH	$\beta_2 n^{1/2} V^{1/2}$	$O(n \log n)$	Beardwood et al. [1959], Johnson et al. [1996]
Çavdar–Sokol	$\alpha_1 \sqrt{n \sigma_x' \sigma_y'} + \alpha_2 \sqrt{n A \sigma_x \sigma_y / (\bar{c}_x \bar{c}_y)}$	$O(n \log n)$	Çavdar and Sokol [2015]
Kwon–Golden–Wasil	$(0.8326 - 0.0011 n + 1.1147 R/n) \sqrt{n A}$	$O(n \log n)$	Kwon et al. [1995]
Daganzo	$0.57 \sqrt{n A}$	$O(n \log n)$	Daganzo [1984]
Chien	$0.98 \sqrt{n A}$	$O(n \log n)$	Chien [1992]

4.2 Datasets

We evaluate on three datasets: a multi-dimensional synthetic set, a 2D diverse synthetic set, and the EUC_2D subset of TSPLIB95. The synthetic sets are held-out test splits from our generation pipeline (Appendix B); TSPLIB95 is real-world. The evaluation pipeline assumes symmetric metric distances. Because $L_{\text{TSP}} \leq 2 L_{\text{MST}}$ holds for any metric TSP (double-tree bound), we drop instances with significant asymmetry and violations of the triangle inequality. Under this rule the only TSPLIB95 instance currently excluded is the bridge graph brg180, whose published optimum produces a value of $L_{\text{TSP}}/L_{\text{MST}} \approx 160$.

4.2.1 Multi-dimensional synthetic

16,920 held-out instances with $n \in [5, 1000]$ and $d \in \{2, 3, \dots, 50, 100\}$, sampled by stratified dimension–size–grid buckets across 16 per-dimension distribution types drawn from the generators used for our training set: uniform, normal, clustered (k -means [Lloyd, 1982]), and autoregressive-correlated. Stratification enforces zero overlap with the training split. $d = 100$ is outside the training range and serves as the extrapolation test. The full (d, n) count grid is given in Appendix B.1 (Table 4).

4.2.2 2D diverse synthetic

2,580 instances, $d = 2$, $n \in [5, 1000]$, coordinate grid side-length $G \in [1000, 10000]$. Five distribution classes (Table 2) each probe a different assumption made by classical 2D estimators. Four classes (isotropic, biased, geometric, clustered) follow conventions used in the TSP-instance-generation literature, including the variants reported by Çavdar and Sokol [2015] for the distribution-free baseline. The fifth class, Line Noise, generates near-linear manifolds of the form $y = mx + b + \epsilon$ with small ϵ . Near-collinear point sets are a standard degenerate TSP test case [cf. Papadimitriou, 1977]; we include this class precisely to measure how the model handles linear patterns, where hull-based formulas vanish and bounding-box formulas over-predict. This geometry is not represented in the training distribution.

Table 2: 2D benchmark dataset composition (2,580 instances).

Class	Generators	Count	Stress target
Isotropic [Çavdar and Sokol, 2015, Golden and Stewart, 1985]	Uniform, Normal, Triangular, Truncated Exp.	840	Control: $A_{\text{hull}} \approx A_{\text{box}}$.
Biased / Skewed [Çavdar and Sokol, 2015]	Squeezed, Uni-Tri, Tri-Squeezed, Correlated	840	Aspect-ratio distortion: $\sigma_x \neq \sigma_y$.
Geometric Struct. [Reinelt, 1991, Bossek et al., 2019]	Grid, Boundary (hollow), X-Central	630	Lattice regularity, hollow voids.
Clustered [Golden and Stewart, 1985, Bossek et al., 2019]	k -Means ($k \in [5, 20]$)	60	Bimodal edge-weight distribution.
Line Noise [†]	Linear manifold $y = mx + b + \epsilon$	210	Hull/box divergence as data collapses toward 1D.
Total		2,580	

[†] Not represented in training; included to measure generalization to unseen geometries.

4.2.3 TSPLIB95 EUC_2D common subset

78 EUC_2D instances with n from 51 (e1151) to 18,512 (d18512) [Reinelt, 1991]. This is the intersection where every benchmark model is directly applicable: all classical baselines require 2D Euclidean coordinates, and the MST Ratio baseline needs only a distance matrix. The remaining 32 metric TSPLIB95 instances (CEIL_2D, ATT pseudo-Euclidean, GEO great-circle, EXPLICIT-metric) are not natively Euclidean and are evaluated in Section 5 via MDS embedding, which among our benchmarked baselines only GART 2.0 supports. For large instances where the dense $n \times n$ distance matrix would exceed memory, we replace it with a Delaunay-based MST construction [Delaunay, 1934] that uses only $O(n)$ edges.

4.3 Results

Every table below reports six quantities per model row: SDPE with a 95% bootstrap confidence interval (1,000 resamples) as the primary precision metric, MAPE, median $|\% \text{ error}|$, R^2 on tour cost, R_α^2 on the MST-ratio $\alpha = L_{\text{TSP}}/L_{\text{MST}}$ (coefficient of determination of $\hat{\alpha}$ against the ground-truth α), and median prediction time in milliseconds (feature extraction + MST + inference). The per-bucket instance count is listed once in the bucket-label cell rather than repeated per row. We introduce R_α^2 as a complement to R^2 : on high-density or high-dimensional instances the MST length alone explains most of the tour cost variance, so R^2 saturates near 1.0 for any MST-based estimator. R_α^2 measures how well a model predicts the multiplicative correction on top of the MST baseline and is therefore the discriminative signal for comparing MST-based estimators; it is defined only for models that produce a learned $\hat{\alpha}$ (GART 2.0, GART 1.0). Within each bucket, model rows are sorted in ascending order of SDPE, so the strongest estimator on the precision axis appears first.

Each bucket closes with a separate Optimal-solver row whose only populated cell is the Time column. We define the optimal solver time as the shorter of the times from locally running CONCORDE [Applegate et al., 2006] and LKH-3 [Helsgaun, 2017] on a single core, without any timeout, where both solvers report the same tour length (and that tour length matches the published TSPLIB optimum where applicable). When only one solver reaches the verified optimum, that solver’s time is used; when neither is locally tractable (very large n), the bucket is annotated with a citation to published benchmark times. The column reports milliseconds in scientific notation so the six orders of magnitude spanning small- n and large- n instances remain legible and directly comparable to the model times in the same column.

4.3.1 Multi-dimensional benchmark

We present the 16,920-instance multi-dimensional benchmark in two orthogonal slices: by instance size (Table 10) and by dimension (Table 9). Both tables summarize the same underlying data.

GART 2.0 reaches 0.88% MAPE and 1.28% SDPE overall, against 8.54% / 6.84% for MST Ratio and 21.20% / 14.38% for Hilbert (Figure 1). Precision tightens monotonically as n grows (SDPE drops from 1.98% at $n \leq 10$ to 0.55% at $n \in [501, 1000]$) and as d grows (2.97% at $d = 2$ down to 0.61% at $d \in [30, 50]$), consistent with the theoretical convergence $\alpha \rightarrow 1$ in high dimensions [Percus and Martin, 1996]. The $d = 100$ bucket sits outside the training grid (which caps at $d = 50$) and therefore serves as a transfer test: the feature set must remain predictive at a dimension never seen during training. We acknowledge this is extrapolation in the favorable direction— α concentrates as d grows, so the target variance shrinks with dimension; this is therefore not evidence of generalization. SDPE rises modestly from 0.61% at $d \in [30, 50]$ to 0.65% at $d = 100$, while MAPE moves from 0.38% to 1.04%. The two moves together describe a distribution-shift bias rather than a variance blow-up: the error distribution stays tight (SDPE tracks the shrinking target variance) but its center shifts slightly under genuine extrapolation, which is the benign failure mode for a precision-first estimator. MST Ratio (Table 1) has nearly-flat SDPE because its fixed constant does not adapt to instance structure. Hilbert degrades with n because space-filling locality breaks on non-uniform distributions. The favorable-direction extrapolation caveat noted above applies to every $d = 100$ row reported in this section.

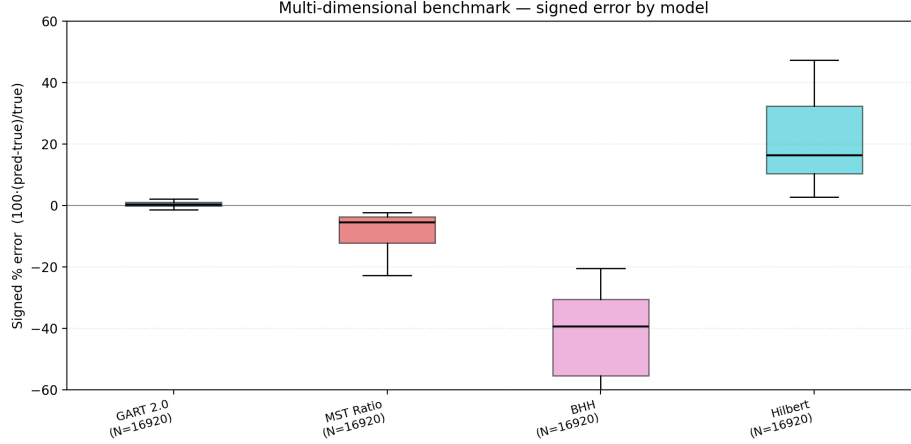


Figure 1: Percent-error distribution per model on the multi-dimensional (ND) benchmark (16,920 instances).

4.3.2 2D diverse benchmark

The 2D diverse benchmark pushes every estimator against five distribution classes that probe distinct failure modes of classical geometry-based estimators. Across the 2,580 instances GART 2.0 holds SDPE under 6.3% in every size bucket and under 3.3% for instances with $n \geq 500$, with SDPE roughly half that of the MST-ratio baseline on the overall aggregate. Figure 2 shows the per-model error distribution, and Table 11 dis-aggregates the results by instance size.

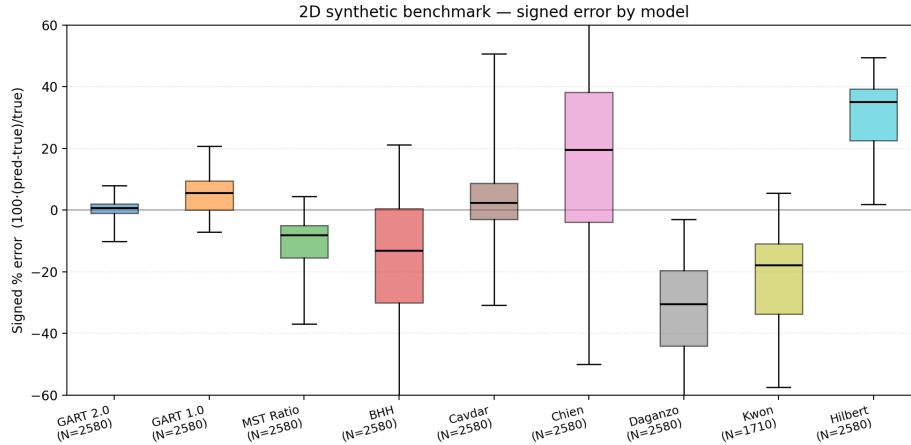


Figure 2: Percent-error distribution per model on the 2D diverse benchmark (2,580 instances).

GART 2.0 leads at 3.33% MAPE / 5.19% SDPE overall, with GART 1.0 [Feldman, 2025] second among ML estimators at 7.35% / 7.95% — the relative improvement over the predecessor model is most pronounced at $n \geq 200$, where GART 2.0’s MST-centric features generalize to larger instances that GART 1.0’s hull-based features cannot track. Among classical baselines, MST Ratio is strongest at 12.45% / 11.06%. The Cavdar–Sokol estimator’s MAPE drops by roughly an order of magnitude, from $\sim 160\%$ in earlier public adaptations of the formula to 16.13% here, with SDPE tightening in parallel from $> 200\%$ to 32.19%; the remaining spread still exceeds Kwon and Daganzo on this benchmark. The gain is traceable to the implementation corrections discussed in Appendix E (rotation into the minimum-area bounding rectangle before evaluating per-axis statistics; the midpoint-axis distance in $\bar{c}_k$ rather than the centroid distance; the convex-hull area as A , following Section 5 of Cavdar and Sokol [2015]). BHH, Chien, and Daganzo all exceed 25% SDPE because their enclosure-based formulas amplify variance on the Biased and Line Noise

classes, where bounding-box and hull volumes diverge. Hilbert has intermediate MAPE (31%) but lower SDPE (14%); its errors are large but consistent — a systematic space-filling bias. Kwon (1995) is structurally invalid above $n \approx 750$ (its linear $-0.0011n$ term forces the estimate negative) and is reported only where applicable.

4.3.3 TSPLIB95 EUC_2D benchmark

Table 12 reports per-bucket results across all 78 EUC_2D instances, with n spanning 51 (e1151) to 18,512 (d18512). Bucket boundaries (23/16/39 instances) separate small, medium, and large real-world instances; the final $n > 400$ bucket aggregates the asymptotic regime where classical MST-ratio calibration approaches its limit.

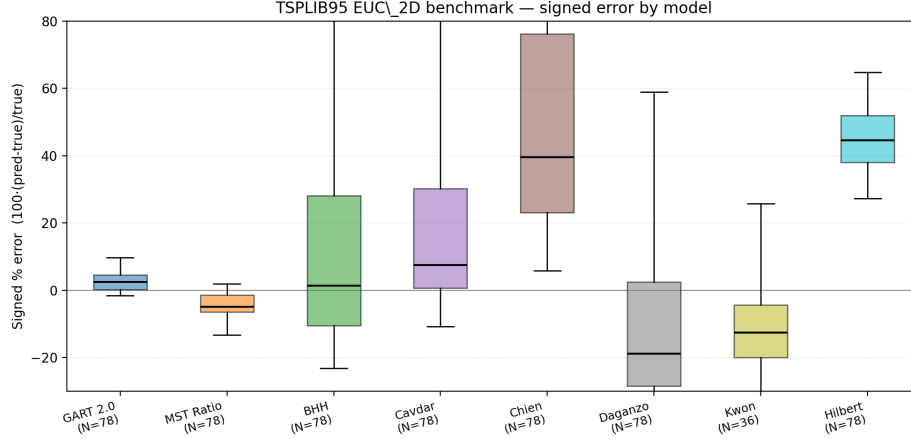


Figure 3: Percent-error distribution per model on the TSPLIB95 EUC_2D subset (78 instances).

Real-world transfer is summarized in Figure 3 and Table 12: GART 2.0 retains the tightest SDPE in every bucket, though the margin collapses to 0.43 percentage points (3.42% vs. 3.85%) in the $n > 400$ bucket where MST Ratio’s fixed constant matches the asymptotic α concentration and the two bootstrap intervals overlap. Overall GART 2.0 leads at SDPE 3.42% / MAPE 3.27% versus SDPE 4.54% / MAPE 5.22% for MST Ratio, the only classical baseline that stays competitive. GART 1.0 [Feldman, 2025], the prior 2D estimator, places second among ML models at SDPE 7.44% / MAPE 8.46%; its convex-hull-dependent features degrade on the largest instances, where GART 2.0’s MST-centric redesign holds steady. Classical baselines BHH, Cavdar–Sokol, Chien, and Daganzo degrade past $n = 400$ as real-world instances deviate from the uniform-density assumptions their derivations require. The largest instance in this subset, d18512 ($n = 18,512$), is more than $18\times$ larger than any training instance and is predicted in under half a second. All 78 EUC_2D instances run end-to-end for every model in this table with the total-time median for that instance across all models still well under a second.

4.4 Discussion

So far we have judged GART 2.0 on pointwise accuracy—how close each predicted tour cost is to the true optimum. Many of the downstream uses motivated in Section 1, however, never consume a single cost value in isolation: they compare instances and act on the comparison, asking which candidate node set is cheaper or which route split to keep. For these uses the estimator must *order* instances the way the true optima would, even when its absolute numbers are slightly off. We therefore close the benchmarking section by measuring this order agreement directly, with two standard rank correlations between predicted and true tour cost: Spearman’s ρ , the correlation of the cost *ranks*, and Kendall’s τ , the fraction of instance pairs the estimator orders concordantly with the truth. Both equal 1 for a perfectly order-preserving estimator and 0 for one unrelated to the truth.

Rank correlations on the same predictions used above confirm strong agreement: GART 2.0 reaches Spearman $\rho = 0.9993$ / Kendall $\tau = 0.981$ on the 2D benchmark, $\rho = 1.0000$ / $\tau = 0.998$ on the multi-dimensional set, and $\rho = 0.9991$ / $\tau = 0.985$ on TSPLIB EUC_2D, outranking MST Ratio, GART 1.0, and Hilbert on both metrics in every case. The precision-first design is therefore also rank-preserving, exactly the property called for in Section 1 of an oracle intended for downstream optimization pipelines.

Global rank correlation near unity is compatible with local pairwise inversions in the dense part of the cost distribution, so we additionally report ε -close pairwise ranking accuracy—the fraction of instance pairs (A, B) with $|L_A - L_B| / \max(L_A, L_B) < \varepsilon$ for which the estimator ranks A, B in the same order as the true optima. At $\varepsilon = 5\%$ GART 2.0 reaches 70.9% on TSPLIB EUC_2D (55 qualifying pairs after excluding instances with ground-truth $\alpha > 2.5$ as degenerate outliers), 72.8% on the 2D synthetic suite (4.5×10^4 pairs), and 89.9% on the multi-dimensional set (2.3×10^4 pairs), versus 61.8%, 71.0%, and 71.7% respectively for the fixed-MST-ratio baseline. At $\varepsilon = 10\%$ these rise to 78.3%/82.0%/94.7%, again dominating the baseline in every slice. This pairwise-local metric is the signal a downstream planner sees when it must choose between two similar-cost candidates, and it is the one on which a global correlation near unity gives the least information; GART 2.0 leads the baseline on it uniformly, which we consider the operational content of the monotone-oracle framing.

This rank agreement is consistent with a classical asymptotic result that also explains when the fixed MST-ratio baseline catches up. For i.i.d. points of bounded density f in $\mathbb{R}^d$, Beardwood et al. [1959] prove that $n^{-(d-1)/d} L_{\text{TSP}}(n) \rightarrow \beta(d) \int f^{(d-1)/d}$ almost surely, and Steele [1988] proves the analogous limit for the MST with its own constant $c(1, d)$ (the Steele theorem states $c(p, d)$ with edge-weight exponent $p = 1$ corresponding to the MST, distinct from the MST-ratio α used elsewhere); dividing the two gives $L_{\text{TSP}}(n)/L_{\text{MST}}(n) \rightarrow \beta(d)/c(1, d)$ almost surely. In the $d = 100$ bucket MST Ratio already sits at 6.55% MAPE against GART 2.0’s 1.04%, and in the largest TSPLIB bucket ($n > 400$) MST Ratio (3.92%) matches GART 2.0 (3.75%) on MAPE with essentially overlapping SDPE bootstrap intervals — exactly the asymptotic regime in which the Beardwood–Steele ratio ceases to vary meaningfully with instance topology. GART 2.0’s advantage is concentrated in the mid-density regime where finite- n deviations from the asymptote dominate, which is also where practical logistics instances live.

The $n > 400$ TSPLIB bucket also exhibits $R_\alpha^2 = -0.549$, which deserves direct acknowledgment rather than a footnote. The coefficient of determination is computed identically in both forms we audit—`sklearn.metrics.r2_score` and the manual expression $1 - \text{SS}_{\text{res}}/\text{SS}_{\text{tot}}$ —and the two agree to six decimal places, so the sign is not a bug. A negative value is legitimate under the sklearn convention whenever the estimator does worse than the per-bucket mean on α , and three ingredients conspire to push R_α^2 below zero on this slice: (i) real-world EUC_2D α concentrates very tightly at large n , $\alpha \in [1.01, 1.20]$ with $\sigma(\alpha) \approx 0.044$, so the denominator SS_{tot} is small; (ii) the model is pulled toward the synthetic-training prior whose mean α at matched n is higher, producing a persistent upward offset; and (iii) the offset is monotone in n —mean $(\hat{\alpha} - \alpha)$ grows from $+0.006$ at $n \leq 100$ to $+0.039$ at $n > 400$, with the fraction of over-predictions rising from 75% to 87% over the same range. The behavior is therefore a systematic distribution-shift bias between synthetic training and real-world TSPLIB geometry, not a variance problem: the SDPE column, which measures the tightness of the error distribution, remains the tightest in the table in every bucket including $n > 400$, because a consistent offset distorts α estimates far less than it distorts the absolute-ratio target R_α^2 . Cost-level accuracy ($R^2 = 1.000$, MAPE 3.75%) is unaffected because L_{MST} dominates L_{TSP} at large n and the multiplicative bias in α becomes a small additive tilt on tour cost. A future revision could correct the offset with a per- n calibration layer; we note the limitation explicitly here rather than hide it in the table.

The classical geometric estimators — BHH, Çavdar–Sokol, Chien, Daganzo — are absent from the ND results by construction: they require either the convex hull area or the bounding-box volume, and hull computation scales as $O(n^{\lfloor d/2 \rfloor})$ [O’Rourke, 1998] while the hull-to-box volume ratio shrinks exponentially with d [Carlsson and Yu, 2024].

The wall-time breakdown reflects the same MST-centric design. On the TSPLIB log — the only timing log that cleanly separates preparation from inference — GART 2.0 spends 84.8% of its wall time on feature extraction and

MST construction and 14.1% on LightGBM inference, while classical baselines spend $\geq 99\%$ on feature and MST preparation. The overhead GART 2.0 introduces relative to any MST-based baseline is therefore the feature pipeline itself, not the tree ensemble; that pipeline cost is sunk as soon as the MST must be built. The remaining weak points are grid lattices and near-linear manifolds (Line Noise) where MST topology becomes structurally degenerate; both configurations are rare in real-world logistics but informative for mapping the model’s boundary.

5 Application: Non-Euclidean Estimation via MDS Embedding

The preceding benchmarks evaluate GART 2.0 on Euclidean point clouds, where the MST is built directly from coordinates. Many real-world instances, however, provide no coordinates at all, or use a non-Euclidean metric that the coordinate-based feature pipeline cannot consume directly. This section extends the estimator to those cases with a multidimensional-scaling pre-step that produces a Euclidean embedding the same trained booster can read, and evaluates it on the non-EUC_2D subset of TSPLIB95, where no classical 2D estimator is even defined.

TSPLIB95 contains 33 instances that are not native EUC_2D: 4 CEIL_2D (Euclidean with ceiling-rounded distances), 2 ATT (pseudo-Euclidean), 10 GEO (great-circle geographic), and 17 EXPLICIT (distance-matrix only). Classical estimators cannot process these: BHH and Çavdar–Sokol require 2D coordinates, Chien requires convex hull area, and Hilbert requires a Euclidean embedding. Among our benchmarked baselines, GART 2.0 is the only one applicable to these cases. Timings are reported as an exploration of capability, not a comparison. We exclude the EXPLICIT outlier brg180 from every aggregate because its distance matrix has 85,368 triangle-inequality violations: GART 2.0 (like any MST-based estimator) assumes symmetric metric distances, so non-metric instances are out of scope for this model. Excluding brg180 leaves 32 metric instances (16 EXPLICIT) for evaluation.

5.1 MDS preprocessing

The following procedure converts any TSPLIB distance matrix into a Euclidean coordinate set that the GART 2.0 feature pipeline can consume unchanged. For each non-Euclidean TSPLIB instance we obtain a Euclidean coordinate representation by classical (metric) multidimensional scaling [Torgerson, 1952, Gower, 1966, Borg and Groenen, 2005, Cox and Cox, 2000]: we double-center the squared distance matrix $D^{(2)}$ and take the leading eigenvectors of the resulting Gram matrix [Gower, 1966]. We use the SciPy eigendecomposition routine [Virtanen et al., 2020] and retain the smallest embedding dimension k that preserves $\geq 99.9\%$ of the positive eigenvalue mass (capped at $k = 100$). A non-Euclidean instance has no native coordinate dimension d , so we write k rather than d for the number of dimensions the embedding produces; this k takes the role that d plays for Euclidean inputs. Geometric features are computed on the k -dimensional MDS embedding while MST features are computed from the original distance matrix D so no metric information is lost, and the selected k is passed to the model as the dimension feature. Full derivation and the per-instance k distribution are given in Appendix L.

5.2 Results on non-Euclidean TSPLIB95

Table 3: GART 2.0 on non-Euclidean TSPLIB95 (32 instances, brg180 excluded).

Distance type	Count	SDPE (%) [95% CI]	MAPE (%)	Median (%)	r_α	Time (ms)	Embed. k (avg [range])
ATT	2	0.48 [—, —]	4.65	4.65	—	23.4	53 [6,100]
CEIL_2D	4	6.12 [0.74, 7.49]	6.49	6.15	+0.656	211	2 [2,2]
GEO	10	2.18 [0.68, 2.97]	3.92	4.57	+0.982	9.3	16 [2,61]
EXPLICIT-metric	16	5.90 [3.78, 7.43]	6.22	5.46	+0.868	7.6	37 [8,100]
TOTAL	32	5.56 [3.86, 6.99]	5.43	4.99	+0.868	9.2	—

The embedded dimension k is chosen adaptively per instance by the MDS variance-retention rule, and r_α is undefined for the ATT row because only two instances are available. No CONCORDE or LKH-3 solver time is reported: neither

solver is natively applicable to non-Euclidean distance types without format-specific translation, which falls outside the scope of this experiment. As reported in Table 3, GART 2.0 generalizes across all four distance types at an aggregate SDPE of 5.56% and MAPE of 5.43% over the 32 instances, with the per-type MAPE spanning 3.92–6.49%. GEO instances achieve the lowest error (MAPE 3.92%) because great-circle distances admit a low-stress Euclidean embedding at typical geographic scales. EXPLICIT-metric instances reach 6.22% because their matrices may mildly violate the triangle inequality, inflating MDS stress. The CEIL_2D row’s comparatively high 211 ms time should not be read as a typical per-instance cost: that group contains only four instances, two of them larger than $n = 10,000$, so its median is dominated by those two large cases rather than representative of the distance type.

6 Conclusion

We presented GART 2.0, a TSP tour-length estimator that predicts the MST-to-TSP scaling factor α using 30 dimension-agnostic features combining MST topology with lightweight coordinate-range descriptors. Across 2,580 synthetic 2D instances, 16,920 multi-dimensional test instances spanning $d = 2$ to $d = 100$, and 110 metric TSPLIB95 instances (78 EUC_2D and 32 non-Euclidean via MDS), MST-topology features recover the per-instance scaling factor with high precision and monotonicity across dimensions and distribution types, without any enclosure geometry. The primary contribution is a monotone, precision-first estimator: its SDPE is materially tighter than the MST-ratio baseline and its error ordering tracks the true cost ordering, so it can serve as a rank-preserving oracle inside a larger optimization pipeline. It outperforms the MST-ratio baseline on every benchmark, including the 78 EUC_2D TSPLIB95 instances whose largest $n = 18,512$ is more than $18\times$ the training cap; the Delaunay-accelerated feature pipeline yields sub-500 ms prediction at $n = 1,000$. Because a single MDS-hybrid pipeline lets GART 2.0 handle every metric TSPLIB95 distance type—EUC_2D, CEIL_2D, ATT, GEO, and EXPLICIT—it applies to a broader range of real-world instances than the classical estimators we benchmark, which are restricted to 2D Euclidean inputs.

Three limitations point at concrete next steps. The α regressor is not yet tight: the coefficient of determination on α and related diagnostics in the benchmarking tables show that a non-trivial share of the target variance is still unexplained, and a more expressive regressor such as a deeper LightGBM, a small neural head on the current features, or a probabilistic model targeting the conditional distribution of α should reduce SDPE further without new feature engineering. Feature extraction time still scales with d even though feature definitions do not, because pairwise distances, centroid statistics, and MST construction all scale with d ; a principled dimension-reduction front-end such as a learned embedding or random projection would make dimension-independence hold in cost as well as in definition. At large n under near-uniform spatial density, the true α concentrates tightly around its asymptotic BHH value and a constant- α estimator is already comparable to GART 2.0 at lower cost; GART 2.0’s precision advantage is strongest at lower n and under distributions that deviate from uniform, so a dispatch layer that runs the MST-ratio baseline in the easy regime and defers to GART 2.0 otherwise is a natural follow-on. A separate next step is to embed the estimator inside a larger optimization loop, such as a Vehicle Routing Problem sub-tour estimator [Mariescu-Istodor and Fränti, 2021] or a bounding function in branch-and-bound, to measure whether the precision advantage translates into faster, more reliable end-to-end decisions.

References

- Richa Agarwala, David L Applegate, Donna Maglott, Gregory D Schuler, and Alejandro A Schäffer. A fast and scalable radiation hybrid map construction and integration strategy. *Genome Research*, 10(3):350–364, 2000.
- Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 2623–2631. ACM, 2019. doi: 10.1145/3292500.3330701.
- David L. Applegate, Robert E. Bixby, Vašek Chvátal, and William J. Cook. *The Traveling Salesman Problem: A Computational Study*. Princeton University Press, 2006.
- Florin Avram and Dimitris Bertsimas. The minimum spanning tree constant in geometrical probability and under the independent model: a unified approach. *The Annals of Applied Probability*, 2(1):113–130, 1992. doi: 10.1214/aoap/1177005773.
- John J. Bartholdi, III and Loren K. Platzman. An $O(N \log N)$ planar travelling salesman heuristic based on spacefilling curves. *Operations Research Letters*, 1(4):121–125, 1982. doi: 10.1016/0167-6377(82)90034-6.
- John Basel and Thomas R. Willemain. Random-tour techniques for estimating optimal traveling salesman tour lengths. *European Journal of Operational Research*, 128(3):537–549, 2001.
- Jillian Beardwood, John H. Halton, and John Michael Hammersley. The shortest path through many points. *Mathematical Proceedings of the Cambridge Philosophical Society*, 55:299–327, 1959. doi: 10.1017/S03050004100034095.
- Ingwer Borg and Patrick J.F. Groenen. *Modern Multidimensional Scaling: Theory and Applications*. Springer, 2nd edition, 2005. doi: 10.1007/0-387-28981-X.
- Jakob Bossek, Pascal Kerschke, Aneta Neumann, Markus Wagner, Frank Neumann, and Heike Trautmann. Evolving diverse TSP instances by means of novel and creative mutation operators. In *Proc. 15th ACM/SIGEVO Conference on Foundations of Genetic Algorithms (FOGA XV)*, pages 58–71, 2019. doi: 10.1145/3299904.3340307.
- Jean Bourgain. On Lipschitz embedding of finite metric spaces in Hilbert space. *Israel Journal of Mathematics*, 52(1–2):46–52, 1985. doi: 10.1007/BF02776078.
- Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.
- John Gunnar Carlsson and Keyi Yu. A new upper bound for the Euclidean TSP constant. *INFORMS Journal on Computing*, 2024. doi: 10.1287/ijoc.2024.0538.
- Bahar Çavdar and Joel Sokol. A distribution-free TSP tour length estimation model for random graphs. *European Journal of Operational Research*, 243(2):588–598, 2015. doi: 10.1016/j.ejor.2014.12.020.
- Nicolas J. Cerf, Jean Boutet de Monvel, Oriol Bohigas, Olivier C. Martin, and Allon G. Percus. The random link approximation for the Euclidean traveling salesman problem. *Journal de Physique I*, 7(1):117–136, 1997. doi: 10.1051/jp1:1997130.
- Bernard Chazelle. An optimal convex hull algorithm in any fixed dimension. *Discrete & Computational Geometry*, 10(4):377–409, 1993. doi: 10.1007/BF02573985.
- T William Chien. Operational estimators for the length of a traveling salesman tour. *Computers & Operations Research*, 19(6):469–478, 1992. doi: 10.1016/0305-0548(92)90002-M.
- Nicos Christofides. Worst-case analysis of a new heuristic for the travelling salesman problem. Technical Report 388, Carnegie-Mellon University, 1976. Reprinted in *Operations Research Forum*, 3:20, 2022.
- William Cook and André Rohe. Computing minimum-weight perfect matchings. *INFORMS Journal on Computing*, 11(2):138–148, 1999.
- Mario Cortina-Borja and Tom Robinson. Estimating the asymptotic constants of the total length of Euclidean minimum spanning trees with power-weighted edges. *Statistics & Probability Letters*, 47(2):125–128, 2000. doi: 10.1016/S0167-7152(99)00149-2.

- Trevor F. Cox and Michael A.A. Cox. *Multidimensional Scaling*. Chapman and Hall/CRC, 2nd edition, 2000. ISBN 9781584880943.
- Carlos F. Daganzo. The distance traveled to visit N points with a maximum of C stops per vehicle: An analytic model and an application. *Transportation Science*, 18(4):331–350, 1984. doi: 10.1287/trsc.18.4.331.
- Boris Delaunay. Sur la sphère vide. *Bulletin de l’Académie des Sciences de l’URSS, Classe des Sciences Mathématiques et Naturelles*, 6:793–800, 1934.
- Doratha E. Drake and Stefan Hougardy. A simple approximation algorithm for the weighted matching problem. *Information Processing Letters*, 85(4):211–213, February 2003. doi: 10.1016/S0020-0190(02)00393-9.
- Ran Duan and Seth Pettie. Linear-time approximation for maximum weight matching. *Journal of the ACM (JACM)*, 61(1):1–23, 2014.
- Jack Edmonds. Paths, trees, and flowers. *Canadian Journal of Mathematics*, 17:449–467, 1965. doi: 10.4153/CJM-1965-045-4.
- Stephen Feldman. Tour length estimation guided vehicle routing. Master’s thesis, Rensselaer Polytechnic Institute, 2025. URL <https://search.proquest.com/openview/d4a8aaeb70d19bb62af72359b5caf7f7/1?pq-origsite=gscholar&cbl=18750&diss=y>. Introduces the original GART 1.0 tour-length estimator. Available via ProQuest Dissertations Publishing.
- Finmile. Revolutionizing last-mile delivery for unparalleled efficiency and customer satisfaction. Whitepaper, Finmile, 2025. URL <https://finmile.co/resources/finmile-revolutionizing-last-mile-delivery>.
- Herbert Freeman and Ruth Shapira. Determining the minimum-area encasing rectangle for an arbitrary closed curve. *Communications of the ACM*, 18(7):409–413, 1975. doi: 10.1145/360881.360919.
- Jerome H. Friedman. Greedy function approximation: A gradient boosting machine. *Annals of Statistics*, 29(5):1189–1232, 2001.
- Harold N. Gabow. Data structures for weighted matching and nearest common ancestors with linking. In *Proceedings of the 1st Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 434–443, 1990.
- Bruce L. Golden and William R. Stewart. Empirical analysis of heuristics. In E. L. Lawler, J. K. Lenstra, A. H. G. Rinnooy Kan, and D. B. Shmoys, editors, *The Traveling Salesman Problem: A Guided Tour of Combinatorial Optimization*, pages 207–249. Wiley, 1985.
- J. C. Gower. Some distance properties of latent root and vector methods used in multivariate analysis. *Biometrika*, 53(3–4):325–338, 1966. doi: 10.1093/biomet/53.3-4.325.
- Russell Halper and Srinivasa Raghavan. The mobile facility routing problem. *Transportation Science*, 45(3):413–434, 2011. doi: 10.1287/trsc.1100.0335.
- Keld Helsgaun. An extension of the Lin-Kernighan-Helsgaun TSP solver for constrained traveling salesman and vehicle routing problems. Technical report, Roskilde University, 2017.
- Chuck Holland, Jack Levis, Ranganath Nugehalli, Bob Santilli, and Jeff Winters. UPS optimizes delivery routes. *Interfaces*, 47(1):8–23, 2017. doi: 10.1287/inte.2016.0875. Winner of the 2016 Franz Edelman Award.
- David S. Johnson, Lyle A. McGeoch, and Edward E. Rothberg. Asymptotic experimental analysis for the Held–Karp traveling salesman bound. In *Proceedings of the 7th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 341–350, 1996.
- I.T. Jolliffe. *Principal Component Analysis*. Springer, 2002.
- Anna R. Karlin, Nathan Klein, and Shayan Oveis Gharan. A (slightly) improved approximation algorithm for metric TSP. In *Proceedings of the 53rd Annual ACM Symposium on Theory of Computing*, pages 32–45, 2021. doi: 10.1145/3406325.3451009.

- Richard M. Karp. Reducibility among combinatorial problems. In Raymond E. Miller and James W. Thatcher, editors, *Complexity of Computer Computations*, pages 85–103. Plenum Press, 1972. doi: 10.1007/978-1-4684-2001-2_9.
- Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. Lightgbm: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems*, pages 3146–3154, 2017.
- Shuhan Kou, Bruce Golden, and Stefan Poikonen. Optimal TSP tour length estimation using standard deviation as a predictor. *Computers & Operations Research*, 148:105993, 2022. doi: 10.1016/j.cor.2022.105993.
- Shuhan Kou, Bruce Golden, and Stefan Poikonen. Optimal TSP tour length estimation using Sammon maps. *Optimization Letters*, 17(1):89–105, 2023. doi: 10.1007/s11590-022-01937-y.
- Shuhan Kou, Bruce Golden, and Stefan Poikonen. Estimating optimal objective values for the TSP, VRP, and other combinatorial problems using randomization. *International Transactions in Operational Research*, 31:3443–3458, 2024. doi: 10.1111/itor.13260.
- Ojeong Kwon, Bruce L. Golden, and Edward A. Wasil. Estimating the length of the optimal TSP tour: An empirical study using regression and neural networks. *Computers & Operations Research*, 22(10):1039–1046, 1995. doi: 10.1016/0305-0548(94)00092-L.
- Siu Kwan Lam, Antoine Pitrou, and Stanley Seibert. Numba: A LLVM-based Python JIT compiler. In *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC*, pages 1–6, 2015. doi: 10.1145/2833157.2833162.
- Stuart Lloyd. Least squares quantization in PCM. *IEEE Transactions on Information Theory*, 28(2):129–137, 1982.
- Scott M. Lundberg and Su-In Lee. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems 30 (NeurIPS)*, pages 4765–4774, 2017.
- Radu Marinescu-Istodor and Pasi Fränti. Solving the large-scale TSP problem in 1 h: Santa Claus Challenge 2020. *Frontiers in Robotics and AI*, 8:689908, 2021. doi: 10.3389/frobt.2021.689908.
- Joseph O’Rourke. *Computational Geometry in C*. Cambridge University Press, 2nd edition, 1998.
- Xuanhao Pan, Yan Jin, Yuandong Ding, Mingxiao Feng, Li Zhao, Lei Song, and Jiang Bian. H-TSP: Hierarchically solving the large-scale traveling salesman problem. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pages 9345–9353, Washington, DC, USA, 2023. AAAI Press. doi: 10.1609/aaai.v37i8.26120.
- Christos H. Papadimitriou. The Euclidean traveling salesman problem is NP-complete. *Theoretical Computer Science*, 4(3):237–244, 1977. doi: 10.1016/0304-3975(77)90012-3.
- Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- Allon G. Percus and Olivier C. Martin. Finite size and dimensional dependence in the Euclidean traveling salesman problem. *Physical Review Letters*, 76(8):1188–1191, 1996. doi: 10.1103/PhysRevLett.76.1188.
- Robert Clay Prim. Shortest connection networks and some generalizations. *Bell System Technical Journal*, 36(6): 1389–1401, 1957.
- Gerhard Reinelt. TSPLIB—a traveling salesman problem library. *ORSA Journal on Computing*, 3(4):376–384, 1991. doi: 10.1287/ijoc.3.4.376.
- Daniel J. Rosenkrantz, Richard E. Stearns, and Philip M. Lewis, II. An analysis of several heuristics for the traveling salesman problem. *SIAM Journal on Computing*, 6(3):563–581, 1977. doi: 10.1137/0206041.
- Anatoliy I. Serdyukov. On some extremal tours in graphs (in Russian). *Upravlyaemye Sistemy*, 17:76–79, 1978.
- Kate Smith-Miles and Jano van Hemert. Towards objective measures of algorithm performance across instance space. *Computers & Operations Research*, 37(5):850–862, 2010.

- J. M. Steele. Growth rates of Euclidean minimum spanning trees with power weighted edges. *Annals of Probability*, 16(4):1767–1787, 1988. doi: 10.1214/aop/1176991590.
- Warren S. Torgerson. Multidimensional scaling: I. Theory and method. *Psychometrika*, 17(4):401–419, 1952. doi: 10.1007/BF02288916.
- Taha Varol, Okan Örsan Özener, and Erinc Albey. Neural network estimators for optimal tour lengths of traveling salesperson problem instances with arbitrary node distributions. *Transportation Science*, 58(1):45–66, 2024. doi: 10.1287/trsc.2022.0015.
- Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, et al. SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods*, 17:261–272, 2020. doi: 10.1038/s41592-019-0686-2.
- Guiling Wang, Guohong Cao, Thomas La Porta, and Wensheng Zhang. Mobile sink routing for data collection in wireless sensor networks. *IEEE Transactions on Mobile Computing*, 7(6):756–770, 2008. doi: 10.1109/TMC.2007.70734.

A Code and Data Availability

Our implementation, trained model weights, and benchmark scripts are available at <https://github.com/sfeldmanMIG25/Area-and-Distribution-Free-Estimator-for-TSP>. Synthetic instances can be regenerated using the provided scripts with fixed random seeds.

The GART 2.0 pipeline is implemented in Python using SciPy [Virtanen et al., 2020] for Delaunay triangulation [Delaunay, 1934] and MST construction (via `scipy.sparse.csgraph.minimum_spanning_tree`), Numba [Lam et al., 2015] for JIT-compiled centroid distance computation, and scikit-learn [Pedregosa et al., 2011] for classical MDS embedding of non-Euclidean distance matrices. For instances without Euclidean coordinates (e.g., TSPLIB GEO and EXPLICIT types), MDS projects the distance matrix into Euclidean coordinates for geometric feature extraction while MST features are computed from the original distances; details are given in Appendix L and evaluated in Section 5. The LightGBM model [Ke et al., 2017] is trained and served via the Python API. All benchmarks were run on a single machine (Intel i7 13th Gen, 64 GB RAM, Windows 11). Concorde version 03.12.19 and LKH-3 version 3.0.13 were used for ground-truth computation.

B Training Data Generation Details

The training dataset was constructed by generating synthetic TSP instances across four distribution classes, with dimensions independently assigned per axis:

The uniform class draws coordinates from $\mathcal{U}(0, G)$ where G is the grid size. The normal class draws from $\mathcal{N}(G/2, G/6)$ truncated to $[0, G]$. The clustered class places $k \in [3, 15]$ cluster centers uniformly and draws points from $\mathcal{N}(\mu_k, \sigma_k)$ around each center. The correlated class uses autoregressive structure across dimensions, $x_{d+1} = \rho \cdot x_d + \epsilon$, producing axis-aligned anisotropy.

Each instance’s dimensions are independently assigned a distribution class, producing anisotropic configurations where topology varies by axis. Node counts span $n \in [5, 1,000]$ with training-visible dimensions $d \in \{2, 3, 4, 5, 6, 7, 8, 9, 10, 15, 20, 25, 30, 35, 40, 45, 50\}$ (17 dimension values), plus a held-out extrapolation bucket at $d = 100$ (5,904 instances, all placed in the test split). Ground-truth solutions were computed by running both Concorde and LKH-3, retaining the shorter result.

The 106,272 solved instances are partitioned into training (69,768, 65.6%), validation (19,584, 18.4%), and test (16,920, 15.9%) using a stratified sample on $(d, n, \text{grid size})$. The stratification rule ranks instances within each stratum by a seeded uniform and assigns roughly the first 16% to test, the next 18% to validation, and the remainder to training; $d = 100$ instances bypass this rule and are locked to the test partition (see `feature_creator_v3.create_stratified_split`). Table 5 lists the per- (d, n) training-instance count; grid size $G \in \{100, 1,000, 10,000\}$ is sampled uniformly within each cell.

B.1 Multi-dimensional benchmark grid

For reference, the full (d, n) count grid of the 16,920-instance ND benchmark:

Table 4: Multi-dimensional synthetic benchmark grid: instance count per (d, n) bucket (16,920 instances total).

$d \setminus n$	[5, 10]	[11, 50]	[51, 100]	[101, 200]	[201, 500]	[501, 1000]	Total
$d = 2$	162	108	135	27	81	135	648
$d = 3-5$	486	324	405	81	243	405	1,944
$d = 6-10$	810	540	675	135	405	675	3,240
$d = 15-25$	486	324	405	81	243	405	1,944
$d = 30-50$	810	540	675	135	405	675	3,240
$d = 100$	1,476	984	1,230	246	738	1,230	5,904
Total	4,230	2,820	3,525	705	2,115	3,525	16,920

Table 5: Training-set instance count per (d, n) cell (69,768 total; grid size $G \in \{100, 1,000, 10,000\}$ sampled uniformly within each cell; four distribution classes per axis listed in §B).

$d \setminus n$	[5, 10]	[11, 50]	[51, 100]	[101, 200]	[201, 500]	[501, 1000]	Total
$d = 2$	1,026	684	855	171	513	855	4,104
$d = 3-5$	3,078	2,052	2,565	513	1,539	2,565	12,312
$d = 6-10$	5,130	3,420	4,275	855	2,565	4,275	20,520
$d = 15-25$	3,078	2,052	2,565	513	1,539	2,565	12,312
$d = 30-50$	5,130	3,420	4,275	855	2,565	4,275	20,520
Total	17,442	11,628	14,535	2,907	8,721	14,535	69,768

C Detailed Feature Specifications

Tables 6 and 7 list the 22 MST-derived features used by GART 2.0, complementing the geometric features of Table 13 (§3.1). Listed complexities assume the MST has already been built; MST construction itself dominates the asymptotic cost and is analyzed in §3.

C.1 MST Edge Distribution Statistics

The second category captures the local uniformity of the point set through the statistical moments of the MST edge weights. Unlike coordinate-based variance, edge weights are invariant to rotation and translation. High skewness or kurtosis in this distribution typically indicates a mixture of scales, such as dense clusters separated by voids, which significantly impacts the α ratio.

Table 6: MST Edge Statistic Specifications

Feature Name	Description & Rationale	Complexity	Origin / Citation
MST Edge Max	Weight of the longest edge in the MST.	$O(n)$	Standard
MST Quantiles	10%, 25%, 50%, 75%, 90% percentiles. Provides a non-parametric distribution profile.	$O(n \log n)$	Proposed ; via MST [Prim, 1957]
MST Total Length	Sum of all edge weights. The primary scaling factor; $L_{TSP} \geq L_{MST}$.	$O(n^2)$	Prim [1957]
MST Edge Mean	Average edge length in the MST.	$O(n)$	Smith-Miles and van Hemert [2010]
MST Edge Std	Standard deviation of edge lengths in the MST.	$O(n)$	Smith-Miles and van Hemert [2010]
MST Edge Skewness	Asymmetry of edge distribution. High positive skew implies many short edges and few long bridges.	$O(n)$	Smith-Miles and van Hemert [2010]
MST Edge Kurtosis	Tailedness of distribution. Indicates the presence of extreme outliers.	$O(n)$	Smith-Miles and van Hemert [2010]

C.2 Topological Structure and Clustering Proxies

The final category contains features designed to detect higher-order structures—such as clusters, filaments, and hubs—without performing expensive clustering algorithms. We introduce specific ratios (Dominance, Gap, Normalized Diameter) that serve as $O(n)$ proxies for these complex geometric properties. For instance, the "Leaf Ratio" effectively distinguishes between low-dimensional linear manifolds (low leaf count) and high-dimensional isotropic noise (high leaf count).

Table 7: Topological and Clustering Feature Specifications

Feature Name	Description & Rationale	Complexity	Origin / Citation
MST Dominance Ratio	Sum of largest $\sqrt{n}$ edges / Total Length. Captures weight of inter-cluster bridges.	$O(n)$	Proposed ; via MST [Prim, 1957]
MST Gap Ratio	Max edge / Median edge. Measures magnitude of sparsity jumps.	$O(1)$	Proposed ; via MST [Prim, 1957]
MST Leaf Ratio	Fraction of nodes with degree $k = 1$. Topology proxy for dimensionality.	$O(n)$	Smith-Miles and van Hemert [2010]
MST Degree (Mean, Std)	Branching factor statistics.	$O(n)$	Smith-Miles and van Hemert [2010]
MST Degree Max	Maximum node degree. Identifies "hub" nodes common in High-D spaces.	$O(n)$	Smith-Miles and van Hemert [2010]
MST Diameter	Sum of edge weights on the longest path between any two leaves, computed via double-BFS.	$O(n)$	Graph Theory
Normalized Diameter	Diameter / Total Length. Scale-invariant shape proxy ($= 1.0$ for path-manifolds exactly).	$O(1)$	Proposed ; via double-BFS
Large Edge Count	Count of edges $> \mu + \sigma$. Secondary cluster indicator.	$O(n)$	Proposed ; via MST [Prim, 1957]

D Feature Importance: SHAP Values

To validate the 30-feature set, we report the SHAP values [Lundberg and Lee, 2017] on the held-out test split using the saved booster (see `shap_analyzer_v3.py`). Table 8 ranks all 30 features by mean absolute SHAP contribution. Three observations motivate the final feature set.

First, the MST dominance ratio alone carries 26.4% of the model’s decision mass, and six of the top ten features are MST-derived (dominance ratio, degree mean, diameter, diameter-normalized, gap ratio, large-edge count). This is consistent with the BHH intuition that tour length scales with tree-like spanning structure; it also justifies the decision to feature-engineer from the MST rather than from raw coordinates. Second, dimension and size are first-class inputs. n and d together contribute 22.2% of the decision mass, which is how the model adapts to varying problem scale without being given a distribution label. Finally, lightweight spread statistics add coverage. Centroid-distance dispersion

(4.8% share on `centroid_dist_std` alone, 9.9% summed across the four centroid-distance descriptors) and bounding hypervolume ($\sim 4.6\%$ summed across the log and linear forms) contribute meaningfully once evaluated in the canonical principal-axis frame (§3.1), which is what allows the rotation-sensitive block to carry signal without leaking orientation noise and without requiring a convex-hull computation, keeping the feature set applicable at arbitrary d .

Additional testing showed that introducing further features found in literature such as a Greedy Nearest Neighbor Tour led to overfitting with the model learning the underlying distributions from the training data rather than generalizing. SHAP analysis was used to iteratively reduce the features chosen to the 30 features discussed in 3.1.

Table 8: All 30 features of GART 2.0 ranked by mean $|\text{SHAP}|$ (held-out test split, $N = 2,000$ sampled rows, seed 42, recomputed on the current trained booster). The bar shows the share of total SHAP mass.

Rank	Feature	Mean $ \text{SHAP} $	Share (%)	
1	<code>mst_dominance_ratio</code>	0.0388	26.43	
2	<code>dimension</code>	0.0171	11.66	
3	<code>n_customers</code>	0.0155	10.55	
4	<code>centroid_dist_std</code>	0.0071	4.81	
5	<code>mst_degree_mean</code>	0.0068	4.62	
6	<code>mst_diameter</code>	0.0063	4.30	
7	<code>mst_diameter_normalized</code>	0.0061	4.18	
8	<code>mst_gap_ratio</code>	0.0050	3.39	
9	<code>large_edge_count</code>	0.0044	2.98	
10	<code>bounding_hypervolume</code>	0.0040	2.70	
11	<code>mst_leaf_ratio</code>	0.0039	2.66	
12	<code>mst_degree_max</code>	0.0034	2.32	
13	<code>centroid_dist_mean</code>	0.0030	2.05	
14	<code>log_bounding_hypervolume</code>	0.0027	1.85	
15	<code>centroid_dist_max</code>	0.0024	1.61	
16	<code>centroid_dist_iqr</code>	0.0022	1.51	
17	<code>mst_edge_max</code>	0.0020	1.35	
18	<code>mst_edge_q75</code>	0.0017	1.19	
19	<code>aspect_ratio</code>	0.0017	1.13	
20	<code>mst_degree_std</code>	0.0016	1.11	
21	<code>log_node_density</code>	0.0015	1.01	
22	<code>mst_edge_q90</code>	0.0014	0.97	
23	<code>mst_edge_q25</code>	0.0014	0.93	
24	<code>mst_edge_mean</code>	0.0012	0.81	
25	<code>node_density</code>	0.0011	0.76	
26	<code>mst_edge_skew</code>	0.0011	0.72	
27	<code>mst_edge_q10</code>	0.0010	0.70	
28	<code>mst_edge_q50</code>	0.0010	0.68	
29	<code>mst_edge_kurtosis</code>	0.0009	0.59	
30	<code>mst_edge_std</code>	0.0006	0.41	

E Benchmark-Model Implementation Notes

This appendix records per-baseline implementation details that were removed from §4.1 for brevity.

BHH constant. For $d = 2$ we use the large-scale empirical estimate $\beta_2 = 0.7124 \pm 0.0002$ of Johnson et al. [1996] from Concorde-verified optima on uniform random point sets; an independent statistical-mechanics study [Percus and Martin, 1996] reports $\beta_2 = 0.7120 \pm 0.0002$ under toroidal boundary conditions. We retain 0.7124 because our benchmark consumes uniform point sets under open (square) boundaries. Cerf et al. [1997] reports

$\beta_3 = 0.6979 \pm 0.0002$ for the 3D case; we do not apply BHH for $d \geq 3$ because the hull-to-bounding-box volume ratio decays exponentially with dimension, making V unstable as an estimator [Carlsson and Yu, 2024].

Çavdar–Sokol. Regression coefficients are $\alpha_1 = 2.791$, $\alpha_2 = 0.2669$ as fitted in Çavdar and Sokol [2015]; A is convex-hull area, σ_k is the standard deviation of coordinates in dimension k , $\bar{c}_k$ is the average absolute distance from each node to the *midpoint axis* of the bounding rectangle in dimension k (i.e. to the line $x_k = (\max_k + \min_k)/2$, not to the coordinate mean), and σ'_k is the standard deviation of those absolute distances. Because the original model was trained on axis-aligned rectangular graphs, we rotate coordinates into the minimum-area bounding rectangle using rotating calipers over the convex hull [Freeman and Shapira, 1975] before computing dispersion. For $n < 1000$ we apply the small- n correction $E/T = 0.9325 e^{0.00005298 n} - 0.2972 e^{-0.01452 n}$ from Çavdar and Sokol [2015, Eq. 4].

Kwon–Golden–Wasil. R is the mean centroid distance, a depot-free adaptation of Kwon’s depot-to-region distance. The linear $-0.0011 n$ term was calibrated for VRP customer counts in the tens-to-hundreds range; extrapolated to n in the thousands it drives the prediction negative, which we clip to zero. To stabilize the formula across TSPLIB’s wide coordinate-scale range, we rescale coordinates so the bounding-box diagonal is 1, evaluate Kwon in that normalized space, and multiply the result by the original diagonal. Both behaviors are visible in Table 12: Kwon tracks the data for $n \leq 400$ and degenerates for $n \geq 1500$.

Daganzo vs. BHH. Structurally Daganzo is BHH with a different constant: the 0.57 coefficient is the CVRP calibration of Daganzo [1984], visibly looser than the BHH/uniform-square value 0.7124. We include both because the literature reports them as distinct benchmarks.

Chien. A is convex-hull area, n stops, p routes; we set $k_1 = 0.98$, $k_2 = 0$, $p = 1$ for single-tour evaluation.

Hilbert curve. The constructive heuristic of Bartholdi and Platzman [1982] originally used a Sierpinski-style space-filling curve; the Hilbert variant we implement here is a later refinement in the same family. Points are sorted by their Hilbert index $h(x)$ and visited in that order.

MST Ratio constants. For $d = 2$ we use $\rho(2) = 1.075$ computed as $\beta_{\text{TSP}}(2)/\beta_{\text{MST}}(2) \approx 0.7124/0.663$ [Johnson et al., 1996, Cortina-Borja and Robinson, 2000], consistent with the analytic lower bound of 0.6008 on $\beta_{\text{MST}}(2)$ from Avram and Bertsimas [1992]. For higher dimensions we use the empirical schedule $\rho(3) = 1.05$ and $\rho(d) = 1.0 + 0.075 \cdot (2/d)$ for $d \geq 4$. This schedule is not taken from prior work; we constructed it to interpolate monotonically between the calibrated $d = 2$ value and the $d \rightarrow \infty$ limit of unity established by Percus and Martin [1996], and treat it as a baseline to be compared against, not as a calibrated MST-to-TSP relationship for arbitrary d . These constants are calibrated for uniform distributions; the elevated SDPE observed for MST Ratio in Table 9 reflects the inability of a fixed constant to adapt to topological variation in non-uniform test instances.

F Per-Dimension Detail for the Multi-Dimensional Benchmark

Table 9 reports the multi-dimensional benchmark broken down by dimension d .

Table 9: ND benchmark by dimension; same 16,920 instances as Table 10.

Bucket	Model	SDPE (%) [95% CI]	MAPE (%)	Median (%)	R^2	R_α^2	Time (ms)
$d = 2$ Count=648	GART 2.0	2.97 [2.72, 3.21]	1.98	1.18	1.000	0.947	144
	MST Ratio	10.51 [9.88, 11.10]	13.09	8.88	0.995	—	45
	Hilbert	15.78 [14.81, 16.79]	29.71	35.19	0.805	—	1.25
	<i>Optimal solver</i>	—	—	—	—	—	102 ms
$d = 3-5$ Count=1,944	GART 2.0	2.03 [1.91, 2.14]	1.30	0.69	1.000	0.951	160
	MST Ratio	7.91 [7.61, 8.18]	11.64	8.24	0.996	—	47
	Hilbert	15.72 [15.24, 16.20]	35.70	40.13	0.732	—	1.62
	<i>Optimal solver</i>	—	—	—	—	—	109 ms
$d = 6-10$ Count=3,240	GART 2.0	1.28 [1.22, 1.34]	0.82	0.45	1.000	0.970	167
	MST Ratio	6.56 [6.36, 6.74]	10.21	7.15	0.997	—	42
	Hilbert	13.83 [13.52, 14.12]	32.53	35.93	0.787	—	10.8
	<i>Optimal solver</i>	—	—	—	—	—	122 ms
$d = 15-25$ Count=1,944	GART 2.0	0.85 [0.80, 0.90]	0.54	0.30	1.000	0.984	176
	MST Ratio	6.02 [5.80, 6.23]	8.68	5.63	0.998	—	45
	Hilbert	11.73 [11.46, 12.00]	24.60	26.36	0.844	—	41
	<i>Optimal solver</i>	—	—	—	—	—	123 ms
$d = 30-50$ Count=3,240	GART 2.0	0.61 [0.58, 0.64]	0.38	0.21	1.000	0.991	175
	MST Ratio	5.96 [5.80, 6.13]	7.65	4.59	0.999	—	47
	Hilbert	8.00 [7.86, 8.15]	17.11	18.19	0.925	—	55
	<i>Optimal solver</i>	—	—	—	—	—	136 ms
$d = 100$ Count=5,904	GART 2.0	0.65 [0.64, 0.67]	1.04	0.97	1.000	0.973	178
	Hilbert	4.60 [4.54, 4.66]	10.40	11.07	0.971	—	99
	MST Ratio	5.93 [5.81, 6.04]	6.55	3.44	0.999	—	50
	<i>Optimal solver</i>	—	—	—	—	—	120 ms
Total Count=16,920	GART 2.0	1.28 [1.24, 1.31]	0.88	0.65	1.000	0.974	171
	MST Ratio	6.84 [6.75, 6.94]	8.54	5.58	0.999	—	47
	Hilbert	14.38 [14.24, 14.52]	21.20	16.21	0.962	—	52
	<i>Optimal solver</i>	—	—	—	—	—	122 ms

G Per-Size Detail for the Multi-Dimensional Benchmark

Table 10 reports the multi-dimensional benchmark broken down by instance-size bucket.

Table 10: ND benchmark by instance-size bucket.

Bucket	Model	SDPE (%) [95% CI]	MAPE (%)	Median (%)	R^2	R_α^2	Time (ms)
[5, 10] Count=4,230	GART 2.0	1.98 [1.90, 2.06]	1.33	0.89	1.000	0.935	116
	MST Ratio	5.31 [5.15, 5.47]	18.80	17.92	0.966	—	4.53
	Hilbert	7.49 [7.17, 7.82]	8.15	5.89	0.995	—	0.66
	<i>Optimal solver</i>	—	—	—	—	—	1 ms
[11, 50] Count=2,820	GART 2.0	1.35 [1.28, 1.42]	1.06	0.90	1.000	0.906	114
	MST Ratio	2.99 [2.85, 3.13]	7.46	6.92	0.996	—	26
	Hilbert	12.41 [11.98, 12.89]	19.59	15.81	0.981	—	11
	<i>Optimal solver</i>	—	—	—	—	—	108 ms
[51, 100] Count=3,525	GART 2.0	1.03 [1.00, 1.05]	0.86	0.61	1.000	0.916	131
	MST Ratio	2.20 [2.12, 2.28]	5.36	4.77	0.998	—	42
	Hilbert	13.54 [13.19, 13.92]	24.12	20.15	0.973	—	32
	<i>Optimal solver</i>	—	—	—	—	—	76 ms
[101, 200] Count=705	GART 2.0	0.61 [0.58, 0.65]	0.55	0.50	1.000	0.954	147
	MST Ratio	1.66 [1.57, 1.77]	4.36	4.00	0.999	—	81
	Hilbert	13.12 [12.62, 13.65]	26.84	23.82	0.965	—	73
	<i>Optimal solver</i>	—	—	—	—	—	224 ms
[201, 500] Count=2,115	GART 2.0	0.55 [0.54, 0.56]	0.51	0.37	1.000	0.953	176
	MST Ratio	1.46 [1.42, 1.49]	4.03	3.82	0.999	—	129
	Hilbert	12.63 [12.40, 12.86]	28.18	26.23	0.960	—	144
	<i>Optimal solver</i>	—	—	—	—	—	886 ms
[501, 1000] Count=3,525	GART 2.0	0.55 [0.54, 0.57]	0.48	0.29	1.000	0.949	272
	MST Ratio	1.33 [1.30, 1.35]	3.82	3.64	0.999	—	334
	Hilbert	12.72 [12.57, 12.88]	29.91	28.50	0.953	—	825
	<i>Optimal solver</i>	—	—	—	—	—	3,799 ms
Total Count=16,920	GART 2.0	1.28 [1.24, 1.31]	0.88	0.65	1.000	0.974	171
	MST Ratio	6.84 [6.75, 6.94]	8.54	5.58	0.999	—	47
	Hilbert	14.38 [14.24, 14.52]	21.20	16.21	0.962	—	52
	<i>Optimal solver</i>	—	—	—	—	—	122 ms

H Per-Size Detail for the 2D Diverse Benchmark

Table 11 reports the 2D diverse benchmark broken down by instance-size bucket.

Table 11: 2D diverse synthetic benchmark.

Bucket	Model	SDPE (%) [95% CI]	MAPE (%)	Median (%)	R^2	R^2_{α}	Time (ms)
[5, 10] Count=360	GART 2.0	5.88 [5.46, 6.27]	4.69	3.73	0.991	0.830	64
	GART 1.0	6.93 [6.27, 7.52]	5.27	4.09	0.984	0.784	31
	Daganzo	7.79 [7.25, 8.28]	67.45	66.16	-0.028	—	3.85
	Hilbert	8.84 [8.11, 9.50]	8.35	5.79	0.965	—	0.18
	BHH	9.74 [9.05, 10.36]	59.32	57.70	0.203	—	3.87
	MST Ratio	9.99 [9.33, 10.65]	28.95	29.17	0.785	—	8.77
	Kwon	11.28 [10.42, 12.06]	50.16	48.60	0.422	—	4.61
	Chien	13.40 [12.47, 14.27]	44.04	41.81	0.545	—	3.82
	Çavdar–Sokol	20.11 [18.79, 21.35]	26.54	23.71	0.756	—	8.43
	<i>Optimal solver</i>	—	—	—	—	—	82 ms
[11, 50] Count=480	GART 2.0	6.29 [5.72, 6.81]	4.49	2.99	0.993	0.679	59
	GART 1.0	6.43 [5.85, 6.96]	4.56	2.94	0.991	0.715	37
	Daganzo	7.43 [6.92, 7.96]	42.70	42.10	0.592	—	4.50
	BHH	9.28 [8.63, 9.93]	28.38	27.64	0.817	—	4.38
	Kwon	9.55 [8.76, 10.29]	19.45	18.38	0.908	—	4.72
	MST Ratio	9.66 [8.86, 10.43]	15.16	13.63	0.949	—	12
	Hilbert	10.37 [9.72, 11.07]	27.18	26.95	0.804	—	0.35
	Çavdar–Sokol	11.27 [10.26, 12.18]	9.88	9.10	0.972	—	10
	Chien	12.77 [11.89, 13.69]	10.03	8.51	0.967	—	4.51
	<i>Optimal solver</i>	—	—	—	—	—	85 ms
[51, 100] Count=630	GART 2.0	5.91 [5.32, 6.50]	3.70	2.09	0.995	0.597	80
	GART 1.0	7.38 [6.79, 8.01]	5.39	3.52	0.987	0.494	43
	MST Ratio	8.59 [7.82, 9.36]	11.45	9.34	0.975	—	16
	Hilbert	9.82 [9.11, 10.53]	34.23	34.08	0.734	—	0.60
	Daganzo	10.47 [9.56, 11.32]	30.37	31.50	0.735	—	5.28
	Kwon	13.00 [11.93, 14.09]	13.14	11.59	0.936	—	5.59
	BHH	13.08 [11.97, 14.20]	16.22	14.98	0.912	—	5.16
	Chien	18.00 [16.36, 19.63]	22.06	17.77	0.887	—	5.13
	Çavdar–Sokol	19.50 [17.53, 21.43]	13.59	7.98	0.945	—	11
	<i>Optimal solver</i>	—	—	—	—	—	248 ms
[101, 500] Count=500	GART 2.0	4.48 [3.87, 5.03]	2.57	1.22	0.997	0.535	68
	GART 1.0	5.83 [5.24, 6.45]	9.41	8.51	0.982	-0.903	75
	MST Ratio	6.31 [5.53, 6.96]	7.46	6.06	0.991	—	27
	Hilbert	10.10 [8.94, 11.23]	38.11	38.03	0.722	—	15
	Kwon [†]	19.45 [15.66, 22.74]	27.16	27.51	0.775	—	6.42
	Daganzo	23.93 [20.65, 26.97]	25.59	22.54	0.833	—	5.59
	BHH	29.91 [25.40, 33.86]	14.50	4.70	0.931	—	5.43
	Chien	41.15 [34.93, 46.43]	45.10	35.06	0.674	—	5.31
	Çavdar–Sokol	38.60 [33.11, 43.50]	17.61	3.78	0.923	—	17
	<i>Optimal solver</i>	—	—	—	—	—	2,965 ms
[501, 1000] Count=610	GART 2.0	3.05 [2.73, 3.35]	1.88	0.95	0.997	0.571	100
	MST Ratio	4.54 [4.07, 4.95]	5.71	5.00	0.994	—	45
	GART 1.0	6.38 [5.75, 6.99]	11.09	9.99	0.984	-5.820	108
	Hilbert	11.17 [10.05, 12.30]	38.24	38.21	0.728	—	51
	Daganzo	26.92 [22.92, 30.86]	23.36	17.99	0.850	—	8.05
	BHH	33.64 [28.77, 38.39]	14.75	5.08	0.931	—	8.18
	Chien	46.28 [39.18, 52.99]	49.92	42.71	0.610	—	8.15
	Çavdar–Sokol	41.94 [35.31, 48.05]	16.34	3.32	0.922	—	23
	<i>Optimal solver</i>	—	—	—	—	—	9,032 ms
Total Count=2,580	GART 2.0	5.19 [4.94, 5.42]	3.33	1.78	0.998	0.840	77
	GART 1.0	7.95 [7.65, 8.22]	7.35	6.27	0.988	0.634	50
	MST Ratio	11.06 [10.70, 11.41]	12.45	8.16	0.992	—	19
	Hilbert	14.24 [13.79, 14.66]	31.01	34.99	0.801	—	0.73
	Kwon [†]	20.10 [19.15, 20.87]	24.67	18.97	0.872	—	5.24
	Daganzo	25.64 [24.10, 27.16]	35.25	31.49	0.871	—	5.49
	BHH	32.05 [30.14, 33.99]	23.82	15.55	0.943	—	5.45
	Çavdar–Sokol	32.19 [29.52, 34.85]	16.13	7.09	0.944	—	13
	Chien	44.08 [41.11, 46.66]	33.94	31.21	0.745	—	5.43
	<i>Optimal solver</i>	—	—	—	—	—	459 ms

[†] Kwon's $-0.0011n$ term drives the estimate negative for $n \geq 750$ and is clipped to zero; rows past this threshold are excluded from the Kwon aggregate.

I Per-Size Detail for the TSPLIB95 EUC_2D Benchmark

Table 12 reports the TSPLIB95 EUC_2D benchmark broken down by instance-size bucket.

Table 12: TSPLIB95 EUC_2D benchmark. Time is median wall clock.

n range	Model	SDPE (%) [95% CI]	MAPE (%)	[Median] (%)	R^2	R_α^2	Time (ms)
$n \in [51, 150]$ Count=23	GART 2.0	3.29 [1.76, 4.22]	2.67	1.57	0.996	0.277	9.68
	MST Ratio	4.05 [2.70, 4.84]	6.93	6.12	0.978	—	1.64
	GART 1.0	5.64 [3.58, 7.19]	5.44	3.92	0.988	-1.307	5.29
	Hilbert	11.46 [8.20, 13.91]	39.91	38.92	0.758	—	1.02
	Daganzo	12.80 [6.16, 16.72]	27.94	29.61	0.894	—	0.68
	Çavdar–Sokol	13.91 [7.08, 19.14]	9.59	6.71	0.944	—	1.71
	Kwon	14.27 [6.21, 18.53]	14.83	14.01	0.965	—	0.78
	BHH	15.99 [7.69, 20.89]	16.09	13.59	0.958	—	0.96
	Chien	22.00 [10.58, 28.74]	25.66	21.02	0.700	—	0.68
	<i>Optimal solver</i>	—	—	—	—	—	854 ms
$n \in [151, 400]$ Count=16	GART 2.0	3.46 [2.17, 4.40]	2.96	2.69	0.995	0.516	10
	MST Ratio	5.25 [2.45, 7.17]	5.94	4.93	0.986	—	2.54
	GART 1.0	5.52 [3.87, 6.54]	6.03	5.31	0.985	-0.486	6.31
	Hilbert	7.27 [3.45, 10.25]	44.39	42.42	0.596	—	2.18
	Daganzo	18.63 [9.26, 23.67]	21.33	23.29	0.908	—	0.72
	Kwon	22.77 [13.36, 26.77]	22.41	25.27	0.895	—	0.87
	BHH	23.29 [11.57, 29.59]	16.06	6.53	0.793	—	1.00
	Çavdar–Sokol	27.59 [11.14, 34.47]	20.96	11.34	0.560	—	1.86
	Chien	32.04 [15.91, 40.70]	48.02	36.86	-0.153	—	0.74
	<i>Optimal solver</i>	—	—	—	—	—	15,525 ms
$n > 400$ Count=39	GART 2.0	3.42 [2.68, 3.92]	3.75	2.57	1.000	-0.549	21
	MST Ratio	3.85 [3.06, 4.49]	3.92	3.19	0.998	—	12
	GART 1.0	7.64 [6.07, 8.92]	11.23	9.66	0.978	-10.726	23
	Hilbert	12.55 [9.60, 15.71]	48.66	47.03	0.614	—	12
	Daganzo	39.33 [22.24, 51.03]	25.62	15.21	0.994	—	1.38
	Çavdar–Sokol	44.70 [26.17, 59.49]	33.49	19.61	0.889	—	2.72
	BHH	49.16 [27.80, 63.78]	34.24	15.53	0.877	—	1.81
	Chien	67.62 [38.24, 87.73]	78.22	57.64	0.249	—	1.29
	<i>Optimal solver</i>	—	—	—	—	—	> 1 h
Total Count=78	GART 2.0	3.42 [2.87, 3.89]	3.27	2.52	1.000	0.208	12
	MST Ratio	4.54 [3.58, 5.37]	5.22	4.96	0.998	—	3.61
	GART 1.0	7.44 [6.23, 8.45]	8.46	6.51	0.978	-3.564	8.47
	Hilbert	11.84 [9.83, 13.59]	45.20	44.60	0.623	—	3.59
	Kwon [†]	17.49 [12.84, 20.85]	17.57	16.73	0.939	—	0.79
	Daganzo	32.60 [21.00, 42.89]	25.42	23.68	0.994	—	0.94
	Çavdar–Sokol	36.13 [24.08, 48.14]	23.87	11.07	0.891	—	1.96
	BHH	40.75 [26.25, 53.61]	25.16	13.41	0.880	—	1.13
	Chien	56.05 [36.11, 73.74]	56.53	39.51	0.267	—	0.82
	<i>Optimal solver</i>	—	—	—	—	—	> 1 h

[†] Kwon (1995) was calibrated for VRP customer counts ≈ 50 –300; its linear $-0.0011n$ term drives the estimate negative at large n , where it is clipped to 0 and excluded from this aggregate.

J Geometric and Centroid Feature Specifications

Table 13 lists the 8 geometric and centroid features that complement the MST-derived features in Tables 6, 7.

Table 13: Geometric and Centroid Feature Specifications.

Feature Name	Description & Rationale	Complexity	Origin / Citation
Dimension (d)	The number of spatial coordinates defining each point.	$O(1)$	Standard
Number of Customers (n)	The total count of vertices in the instance.	$O(1)$	Standard
Aspect Ratio	Ratio of the largest dimension range to the smallest. Detects subspace distortion.	$O(n \cdot d)$	Proposed
Bounding Hypervolume	Product of coordinate ranges: $\prod (\max_d - \min_d)$. Approximates volume V for BHH bounds.	$O(n \cdot d)$	Adapted from Beardwood et al. [1959]
Node Density	Ratio of n to Bounding Hypervolume. Proxy for point intensity $\delta = n/V$.	$O(1)$	Proposed ; cf. Beardwood et al. [1959]
Centroid Dist (Mean, Std)	Average and deviation of distances to the centroid. Measures central tendency.	$O(n \cdot d)$	Adapted from Çavdar and Sokol [2015]
Centroid Dist Max	Radius of the enclosing sphere.	$O(n \cdot d)$	Adapted from Çavdar and Sokol [2015]
Centroid Dist IQR	Interquartile range of centroid distances. Robust dispersion metric.	$O(n \cdot d)$	Proposed ; extends Çavdar and Sokol [2015]

K Trained Hyperparameters and Booster Statistics

Table 14 reports the Optuna-tuned hyperparameters and the resulting booster statistics for the final GART 2.0 model.

Table 14: GART 2.0 hyperparameters and trained-model statistics. Tuned block: Optuna TPE, 100 trials, RMSE objective.

Group	Parameter	Value
<i>Tuned hyperparameters (Optuna TPE, 100 trials, RMSE objective)</i>		
<i>Tree structure</i>	num leaves	148
	min child samples	23
	feature fraction	0.548
	bagging fraction / freq	0.609 / 6
<i>Optim. & reg.</i>	learning rate	0.0260
	L1 regularization	7.76×10^{-2}
	L2 regularization	1.19×10^{-5}
<i>Training configuration</i>		
<i>Training setup</i>	boosting type	gbdt
	objective	regression_l2 (RMSE)
	early-stopping patience	100
	random seed	42
<i>Trained-booster statistics</i>		
<i>Booster</i>	trees (after early stop)	2,031
	avg leaves / tree	148.0
	max leaves / tree	148
	avg tree depth	18.89
	max tree depth	33

L MDS Embedding for Non-Euclidean Instances

For TSPLIB instances with non-Euclidean distance types (GEO: geographic coordinates with great-circle distances; EXPLICIT: distance matrix only), we apply classical Multidimensional Scaling (MDS) to obtain a Euclidean coordinate

representation. The embedding dimension k is chosen adaptively to retain at least 99.9% of the positive eigenvalue mass, so that the downstream feature pipeline sees the lowest-dimensional representation that preserves the metric.

Given an $n \times n$ distance matrix D , classical MDS computes the eigendecomposition of the doubly-centered matrix $B = -\frac{1}{2}JD^{(2)}J$ where $J = I - \frac{1}{n}\mathbf{1}\mathbf{1}^T$ and $D^{(2)}$ is the element-wise squared distance matrix. The embedding coordinates are $X = V_k\Lambda_k^{1/2}$ where V_k and Λ_k are the top k eigenvectors and eigenvalues.

We embed into k dimensions, selecting the smallest k that retains $\geq 99.9\%$ of eigenvalue variance (capped at $k = 100$). For GEO instances, the selected dimension ranges from 2 to 61; for EXPLICIT instances, from 8 to 100. The MST features (edge weights, degrees, diameter) are computed from the original distance matrix to preserve metric accuracy, while the geometric features (centroid statistics, bounding volume, aspect ratio) use the MDS embedding. The dimension k is passed to the model as the `dimension` feature.

For GEO instances, the distance function is well-approximated by Euclidean distances at moderate scales, yielding low MDS stress and 3.92% MAPE. For EXPLICIT instances, the distance matrices may violate metric properties (triangle inequality), producing higher stress. After dropping instances that fail the metric-consistency filter from Section 4.2—only `brg180`, whose published optimum exceeds $2 L_{\text{MST}}$ by two orders of magnitude ($L_{\text{TSP}}/L_{\text{MST}} \approx 160$), ruling out any metric interpretation—the remaining 16 EXPLICIT instances achieve 6.22% MAPE.